

Isaac Frimpong Assante

Student ID: 22424646

q1_softmax

Generative AI Use: For the purposes of the assignments, the use of generative AI is subject to the same policies regarding collaboration. Just as with other collaborators, each student must write down the solutions independently of the output of the interaction and the submission should include a note denoting the nature of the collaboration. The use of generative AI tools to substantially complete sections of the assignments is not in line with the spirit of the assignments, and would be a violation of the [Academic honesty, collaboration, AI tools usage](#) of this Course.

```
[1]: # Mounting your Assignment folder

# TODO: Enter the root path on your PC where you have saved the unzipped
# assignment folder, e.g. '/home/cliffbb/Documents/CSCD618-DSCD604/Assignments'
ROOT = r'D:\My Python Environment\Deep Learning And Neural Network\
↳Labs\Assignment'

# TODO: Enter the assignment folder e.g. 'assignment1'
ASSIGNMENT_FOLDER = 'Submit Files'

assert ROOT is not None, "[!] Enter the root path."
assert ASSIGNMENT_FOLDER is not None, "[!] Enter the assignment foldername."

import sys, os
from pathlib import Path
assignment_path = Path(os.path.join(ROOT, ASSIGNMENT_FOLDER))
sys.path.append(assignment_path)

# This downloads the CIFAR-10 dataset to your folder if it doesn't already
↳exist.
%cd "$assignment_path"/csdc618_dscd604/datasets/
!bash get_datasets.sh
%cd "$assignment_path"
```

```
[WinError 3] The system cannot find the path specified: 'D:\\My Python
Environment\\Deep Learning And Neural Network Labs\\Assignment\\Submit
Files
/csd618_dscd604/datasets/'
D:\\My Python Environment\\Deep Learning And Neural Network Labs\\Assignment\\Submit
```

Files

D:\My Python Environment\Deep Learning And Neural Network Labs\Assignment\Submit Files

bash: get_datasets.sh: No such file or directory

1 Softmax Classifier Exercise

In this exercise you will:

- implement a fully-vectorized **loss function** for the Softmax classifier.
- implement the fully-vectorized expression for its **analytic gradient**
- **check your implementation** using numerical gradient
- use a validation set to **tune the learning rate and regularization strength**
- **optimize** the loss function with **SGD**
- **visualize** the final learned weights

```
[2]: # Run some setup code for this notebook.
import random
import numpy as np
from csdc618_dscd604.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

# This is a bit of magic to make matplotlib figures appear inline in the
# notebook rather than in a new window.
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# Some more magic so that the notebook will reload external python modules;
# see http://stackoverflow.com/questions/1907993/
↳ autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2
```

1.1 CIFAR-10 Data Loading and Preprocessing

```
[3]: # Load the raw CIFAR-10 data.
cifar10_dir = 'csdc618_dscd604/datasets/cifar-10-batches-py'

# Cleaning up variables to prevent loading data multiple times (which may cause
↳ memory issue)
try:
    del X_train, y_train
    del X_test, y_test
```

```

    print('Clear previously loaded data.')
except:
    pass

X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

# As a sanity check, we print out the size of the training and test data.
print('Training data shape: ', X_train.shape)
print('Training labels shape: ', y_train.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)

```

```

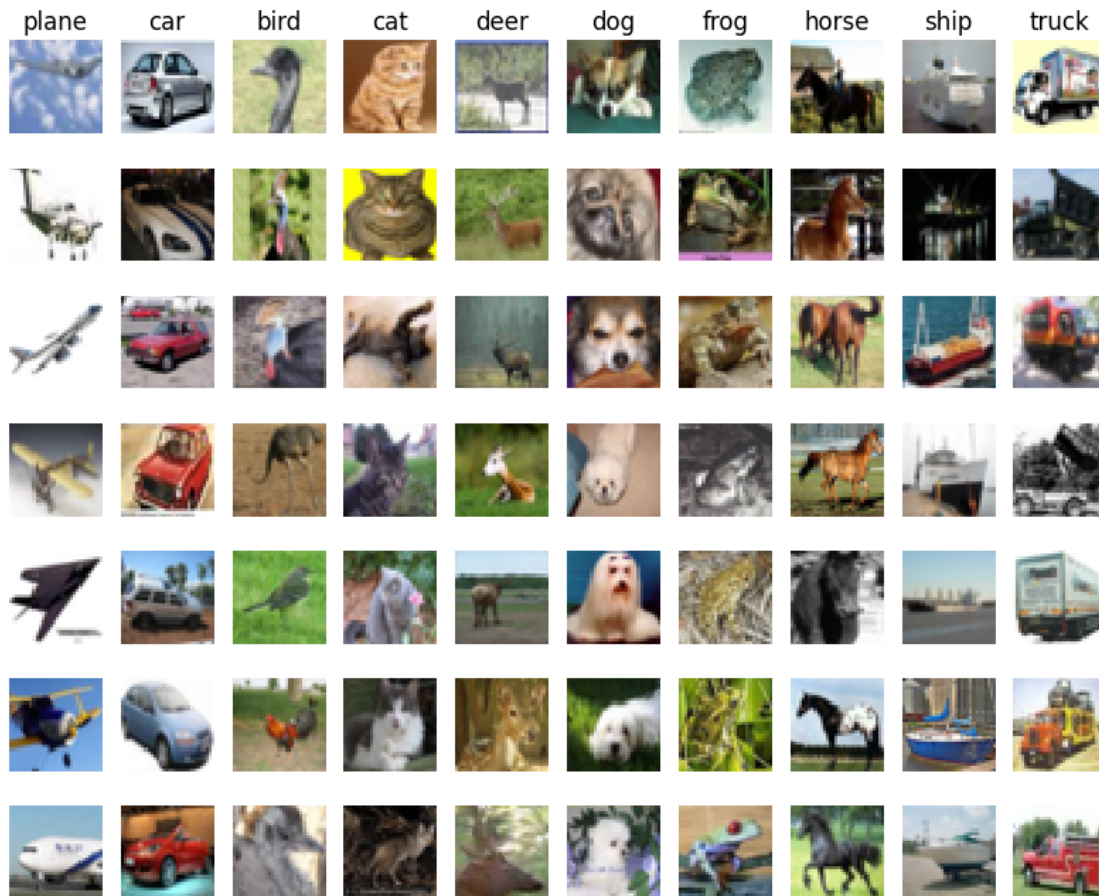
Training data shape: (50000, 32, 32, 3)
Training labels shape: (50000,)
Test data shape: (10000, 32, 32, 3)
Test labels shape: (10000,)

```

```

[4]: # Visualize some examples from the dataset.
# We show a few examples of training images from each class.
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', '
↳ ship', 'truck']
num_classes = len(classes)
samples_per_class = 7
for y, cls in enumerate(classes):
    idxs = np.flatnonzero(y_train == y)
    idxs = np.random.choice(idxs, samples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt_idx = i * num_classes + y + 1
        plt.subplot(samples_per_class, num_classes, plt_idx)
        plt.imshow(X_train[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls)
plt.show()

```



```
[5]: # Split the data into train, val, and test sets. In addition we will
# create a small development set as a subset of the training data;
# we can use this for development so our code runs faster.
num_training = 49000
num_validation = 1000
num_test = 1000
num_dev = 500

# Our validation set will be num_validation points from the original
# training set.
mask = range(num_training, num_training + num_validation)
X_val = X_train[mask]
y_val = y_train[mask]

# Our training set will be the first num_train points from the original
# training set.
mask = range(num_training)
X_train = X_train[mask]
```

```

y_train = y_train[mask]

# We will also make a development set, which is a small subset of
# the training set.
mask = np.random.choice(num_training, num_dev, replace=False)
X_dev = X_train[mask]
y_dev = y_train[mask]

# We use the first num_test points of the original test set as our
# test set.
mask = range(num_test)
X_test = X_test[mask]
y_test = y_test[mask]

print('Train data shape: ', X_train.shape)
print('Train labels shape: ', y_train.shape)
print('Validation data shape: ', X_val.shape)
print('Validation labels shape: ', y_val.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)

```

```

Train data shape: (49000, 32, 32, 3)
Train labels shape: (49000,)
Validation data shape: (1000, 32, 32, 3)
Validation labels shape: (1000,)
Test data shape: (1000, 32, 32, 3)
Test labels shape: (1000,)

```

```

[6]: # Preprocessing: reshape the image data into rows
X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_val = np.reshape(X_val, (X_val.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))
X_dev = np.reshape(X_dev, (X_dev.shape[0], -1))

# As a sanity check, print out the shapes of the data
print('Training data shape: ', X_train.shape)
print('Validation data shape: ', X_val.shape)
print('Test data shape: ', X_test.shape)
print('dev data shape: ', X_dev.shape)

```

```

Training data shape: (49000, 3072)
Validation data shape: (1000, 3072)
Test data shape: (1000, 3072)
dev data shape: (500, 3072)

```

```

[7]: # Preprocessing: subtract the mean image
# first: compute the image mean based on the training data
mean_image = np.mean(X_train, axis=0)

```

```

print(mean_image[:10]) # print a few of the elements
plt.figure(figsize=(4,4))
plt.imshow(mean_image.reshape((32,32,3)).astype('uint8')) # visualize the mean_
    ↳image
plt.show()

# second: subtract the mean image from train and test data
X_train -= mean_image
X_val -= mean_image
X_test -= mean_image
X_dev -= mean_image

# third: append the bias dimension of ones (i.e. bias trick) so that our_
    ↳classifier
# only has to worry about optimizing a single weight matrix W.
X_train = np.hstack([X_train, np.ones((X_train.shape[0], 1))])
X_val = np.hstack([X_val, np.ones((X_val.shape[0], 1))])
X_test = np.hstack([X_test, np.ones((X_test.shape[0], 1))])
X_dev = np.hstack([X_dev, np.ones((X_dev.shape[0], 1))])

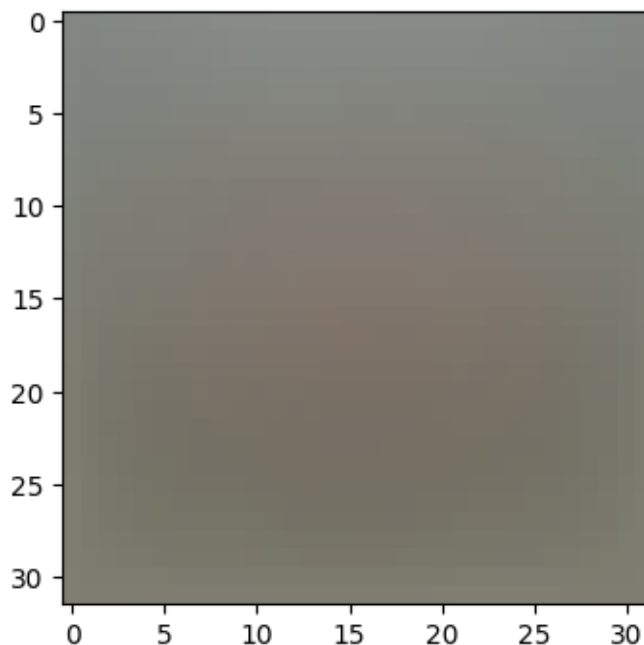
print(X_train.shape, X_val.shape, X_test.shape, X_dev.shape)

```

```

[130.64189796 135.98173469 132.47391837 130.05569388 135.34804082
 131.75402041 130.96055102 136.14328571 132.47636735 131.48467347]

```



```

(49000, 3073) (1000, 3073) (1000, 3073) (500, 3073)

```

1.2 Softmax Classifier

Your code for this section will all be written inside `csdc618_dscd604/classifiers/softmax.py`.

As you can see in the file `csdc618_dscd604/classifiers/softmax.py`, we have prefilled the function `softmax_loss_naive` which uses for loops to evaluate the softmax loss function.

```
[8]: # Evaluate the naive implementation of the loss we provided for you:
from csdc618_dscd604.classifiers.softmax import softmax_loss_naive
import time

# generate a random Softmax classifier weight matrix of small numbers
W = np.random.randn(3073, 10) * 0.0001

loss, grad = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
print('loss: %f' % (loss, ))

# As a rough sanity check, our loss should be something close to -log(0.1).
print('loss: %f' % loss)
print('sanity check: %f' % (-np.log(0.1)))
```

```
loss: 2.387001
loss: 2.387001
sanity check: 2.302585
```

Inline Question 1

Why do we expect our loss to be close to $-\log(0.1)$? Explain briefly.**

Your Answer: At the start the weights are near zero, so all ten class scores are near equal. Softmax turns equal scores into a probability of about 0.1 for each class. The loss equals the negative log probability of the correct class. With probability 0.1 the loss equals $-\log(0.1) \approx 2.3026$.

The `grad` returned from the function the `softmax_loss_naive` is right now **all zero**. Derive and implement the gradient for the softmax loss function and implement it inline inside the function `softmax_loss_naive`. You will find it helpful to interleave your new code inside the existing function.

To check that you have correctly implemented the gradient, you can numerically estimate the gradient of the loss function and compare the numeric estimate to the gradient that you computed. We have provided code that does this for you below:

```
[9]: # Once you've implemented the gradient, recompute it with the code below
# and gradient check it with the function we provided for you

# Compute the loss and its gradient at W.
loss, grad = softmax_loss_naive(W, X_dev, y_dev, 0.0)

# Numerically compute the gradient along several randomly chosen dimensions, and
```

```

# compare them with your analytically computed gradient. The numbers should
↳ match
# almost exactly along all dimensions.
from csdc618_dscd604.gradient_check import grad_check_sparse
f = lambda w: softmax_loss_naive(w, X_dev, y_dev, 0.0)[0]
grad_numerical = grad_check_sparse(f, W, grad)

# do the gradient check once again with regularization turned on
# you didn't forget the regularization gradient did you?
loss, grad = softmax_loss_naive(W, X_dev, y_dev, 5e1)
f = lambda w: softmax_loss_naive(w, X_dev, y_dev, 5e1)[0]
grad_numerical = grad_check_sparse(f, W, grad)

```

```

numerical: 3.991023 analytic: 3.991023, relative error: 5.592648e-09
numerical: -0.470415 analytic: -0.470416, relative error: 1.042505e-07

numerical: 2.682568 analytic: 2.682568, relative error: 3.763492e-09
numerical: -0.007039 analytic: -0.007040, relative error: 9.673192e-06

numerical: 2.561269 analytic: 2.561269, relative error: 1.015019e-08
numerical: 1.593966 analytic: 1.593966, relative error: 1.778411e-08

numerical: -0.727379 analytic: -0.727379, relative error: 2.879090e-08
numerical: 0.198725 analytic: 0.198725, relative error: 8.734972e-08

numerical: 1.978708 analytic: 1.978708, relative error: 1.621912e-08
numerical: -3.437083 analytic: -3.437083, relative error: 1.404528e-10

numerical: 0.192210 analytic: 0.192210, relative error: 6.997423e-08
numerical: 0.838954 analytic: 0.838954, relative error: 7.118395e-08

numerical: 0.568373 analytic: 0.568373, relative error: 1.462740e-07
numerical: 0.785660 analytic: 0.785660, relative error: 1.097857e-08

numerical: -0.876123 analytic: -0.876123, relative error: 1.937934e-08
numerical: -3.382871 analytic: -3.382871, relative error: 2.435236e-09

numerical: 2.868528 analytic: 2.868528, relative error: 5.637204e-09
numerical: -0.131013 analytic: -0.131013, relative error: 1.094790e-08

numerical: 1.607363 analytic: 1.607363, relative error: 5.205409e-08
numerical: 0.769133 analytic: 0.769133, relative error: 4.140806e-09

```

```

[10]: # Next implement the function softmax_loss_vectorized; for now only compute the
↳ loss;
# we will implement the gradient in a moment.
tic = time.time()
loss_naive, grad_naive = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Naive loss: %e computed in %fs' % (loss_naive, toc - tic))

```



```

from csdc618_dscd604.classifiers.softmax import softmax_loss_vectorized
tic = time.time()
loss_vectorized, _ = softmax_loss_vectorized(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Vectorized loss: %e computed in %fs' % (loss_vectorized, toc - tic))

# The losses should match but your vectorized implementation should be much
↪faster.
print('difference: %f' % (loss_naive - loss_vectorized))

```

Naive loss: 2.387001e+00 computed in 0.077558s
Vectorized loss: 2.387001e+00 computed in 0.008193s
difference: 0.000000

```

[11]: # Complete the implementation of softmax_loss_vectorized, and compute the
↪gradient
# of the loss function in a vectorized way.

# The naive implementation and the vectorized implementation should match, but
# the vectorized version should still be much faster.
tic = time.time()
_, grad_naive = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Naive loss and gradient: computed in %fs' % (toc - tic))

tic = time.time()
_, grad_vectorized = softmax_loss_vectorized(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Vectorized loss and gradient: computed in %fs' % (toc - tic))

# The loss is a single number, so it is easy to compare the values computed
# by the two implementations. The gradient on the other hand is a matrix, so
# we use the Frobenius norm to compare them.
difference = np.linalg.norm(grad_naive - grad_vectorized, ord='fro')
print('difference: %f' % difference)

```

Naive loss and gradient: computed in 0.085286s
Vectorized loss and gradient: computed in 0.004027s
difference: 0.000000

1.2.1 Stochastic Gradient Descent

We now have vectorized and efficient expressions for the loss, the gradient and our gradient matches the numerical gradient. We are therefore ready to do SGD to minimize the loss. Your code for this part will be written inside `csdc618_dscd604/classifiers/linear_classifier.py`.

```

[12]: # In the file linear_classifier.py, implement SGD in the function
# LinearClassifier.train() and then run it with the code below.

```

```

from csdc618_dsdc604.classifiers import Softmax
softmax = Softmax()
tic = time.time()
loss_hist = softmax.train(X_train, y_train, learning_rate=1e-7, reg=2.5e4,
                          num_iters=1500, verbose=True)
toc = time.time()
print('That took %fs' % (toc - tic))

```

```

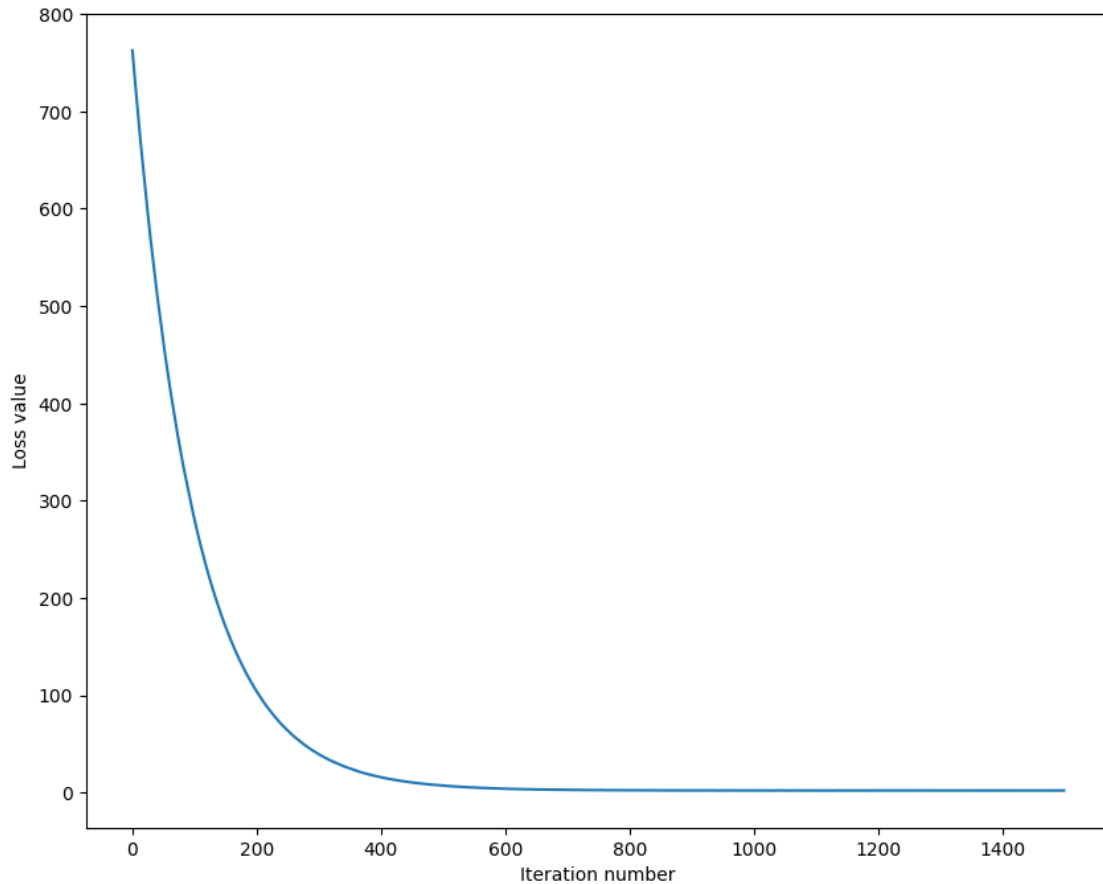
iteration 0 / 1500: loss 762.521605
iteration 100 / 1500: loss 280.081630
iteration 200 / 1500: loss 103.719552
iteration 300 / 1500: loss 39.242765
iteration 400 / 1500: loss 15.737153
iteration 500 / 1500: loss 7.020159
iteration 600 / 1500: loss 3.846690
iteration 700 / 1500: loss 2.750385
iteration 800 / 1500: loss 2.325498
iteration 900 / 1500: loss 2.180637
iteration 1000 / 1500: loss 2.108212
iteration 1100 / 1500: loss 2.111218
iteration 1200 / 1500: loss 2.029400
iteration 1300 / 1500: loss 2.098491
iteration 1400 / 1500: loss 2.117271
That took 7.185950s

```

```

[13]: # A useful debugging strategy is to plot the loss as a function of
      # iteration number:
      plt.plot(loss_hist)
      plt.xlabel('Iteration number')
      plt.ylabel('Loss value')
      plt.show()

```



```
[14]: # Write the LinearClassifier.predict function and evaluate the performance on
# both the training and validation set
# You should get validation accuracy of about 0.34 (> 0.33).
y_train_pred = softmax.predict(X_train)
print('training accuracy: %f' % (np.mean(y_train == y_train_pred), ))
y_val_pred = softmax.predict(X_val)
print('validation accuracy: %f' % (np.mean(y_val == y_val_pred), ))
```

```
training accuracy: 0.327959
validation accuracy: 0.342000
```

```
[15]: # Save the trained model for autograder.
softmax.save("softmax.npy")
```

```
softmax.npy saved.
```

```
[16]: # Use the validation set to tune hyperparameters (regularization strength and
# learning rate). You should experiment with different ranges for the learning
# rates and regularization strengths; if you are careful you should be able to
# get a classification accuracy of about 0.365 (> 0.36) on the validation set.
```

```

# Note: you may see runtime/overflow warnings during hyper-parameter search.
# This may be caused by extreme values, and is not a bug.

# results is dictionary mapping tuples of the form
# (learning_rate, regularization_strength) to tuples of the form
# (training_accuracy, validation_accuracy). The accuracy is simply the fraction
# of data points that are correctly classified.
results = {}
best_val = -1    # The highest validation accuracy that we have seen so far.
best_softmax = None # The Softmax object that achieved the highest validation
    ↪rate.

#####
# TODO:
# Write code that chooses the best hyperparameters by tuning on the validation #
# set. For each combination of hyperparameters, train a Softmax on the.      #
# training set, compute its accuracy on the training and validation sets, and #
# store these numbers in the results dictionary. In addition, store the best  #
# validation accuracy in best_val and the Softmax object that achieves this.  #
# accuracy in best_softmax.
#
# Hint: You should use a small value for num_iters as you develop your      #
# validation code so that the classifiers don't take much time to train; once #
# you are confident that your validation code works, you should rerun the    #
# code with a larger value for num_iters.
#####
# Provided as a reference. You may or may not want to change these
    ↪hyperparameters
learning_rates = [1e-7, 1e-6]
regularization_strengths = [2.5e4, 1e4]
# *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
for lr in learning_rates:
    for reg in regularization_strengths:
        model = Softmax()
        model.train(X_train, y_train, learning_rate=lr, reg=reg,
                    num_iters=2000, verbose=False)
        train_accuracy = np.mean(y_train == model.predict(X_train))
        val_accuracy = np.mean(y_val == model.predict(X_val))
        results[(lr, reg)] = (train_accuracy, val_accuracy)
        if val_accuracy > best_val:
            best_val = val_accuracy
            best_softmax = model
# *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
#####
#                               END OF YOUR CODE
#####

```

```

# Print out results.
for lr, reg in sorted(results):
    train_accuracy, val_accuracy = results[(lr, reg)]
    print('lr %e reg %e train accuracy: %f val accuracy: %f' % (
        lr, reg, train_accuracy, val_accuracy))

print('best validation accuracy achieved during cross-validation: %f' %
      ↪best_val)

```

```

lr 1.000000e-07 reg 1.000000e+04 train accuracy: 0.358122 val accuracy: 0.375000
lr 1.000000e-07 reg 2.500000e+04 train accuracy: 0.326898 val accuracy: 0.340000
lr 1.000000e-06 reg 1.000000e+04 train accuracy: 0.352857 val accuracy: 0.366000
lr 1.000000e-06 reg 2.500000e+04 train accuracy: 0.327245 val accuracy: 0.343000
best validation accuracy achieved during cross-validation: 0.375000

```

```

[17]: # Visualize the cross-validation results
import math
import pdb

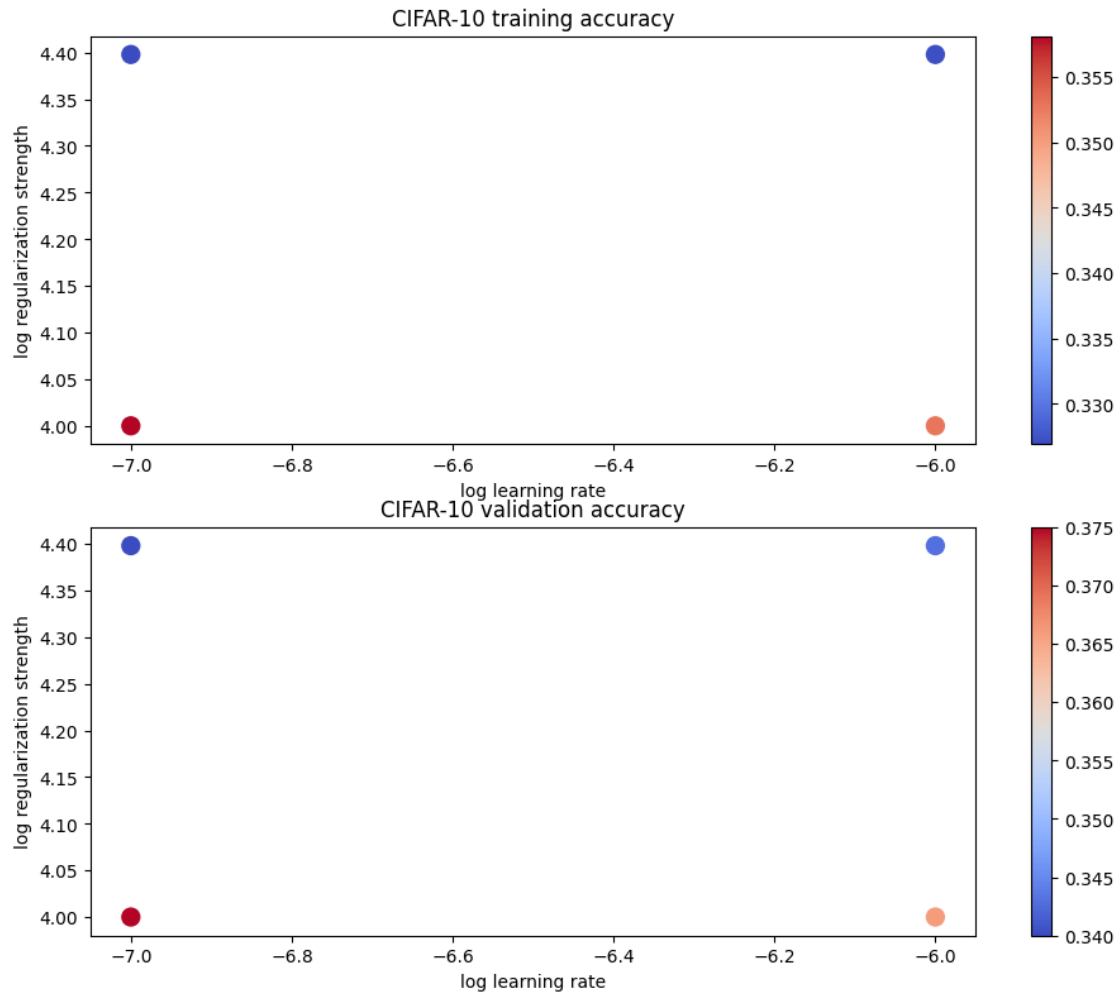
# pdb.set_trace()

x_scatter = [math.log10(x[0]) for x in results]
y_scatter = [math.log10(x[1]) for x in results]

# plot training accuracy
marker_size = 100
colors = [results[x][0] for x in results]
plt.subplot(2, 1, 1)
plt.tight_layout(pad=3)
plt.scatter(x_scatter, y_scatter, marker_size, c=colors, cmap=plt.cm.coolwarm)
plt.colorbar()
plt.xlabel('log learning rate')
plt.ylabel('log regularization strength')
plt.title('CIFAR-10 training accuracy')

# plot validation accuracy
colors = [results[x][1] for x in results] # default size of markers is 20
plt.subplot(2, 1, 2)
plt.scatter(x_scatter, y_scatter, marker_size, c=colors, cmap=plt.cm.coolwarm)
plt.colorbar()
plt.xlabel('log learning rate')
plt.ylabel('log regularization strength')
plt.title('CIFAR-10 validation accuracy')
plt.show()

```



```
[18]: # Evaluate the best softmax on test set
y_test_pred = best_softmax.predict(X_test)
test_accuracy = np.mean(y_test == y_test_pred)
print('Softmax classifier on raw pixels final test set accuracy: %f' % test_accuracy)
```

Softmax classifier on raw pixels final test set accuracy: 0.365000

```
[19]: # Save best softmax model
best_softmax.save("best_softmax.npy")
```

best_softmax.npy saved.

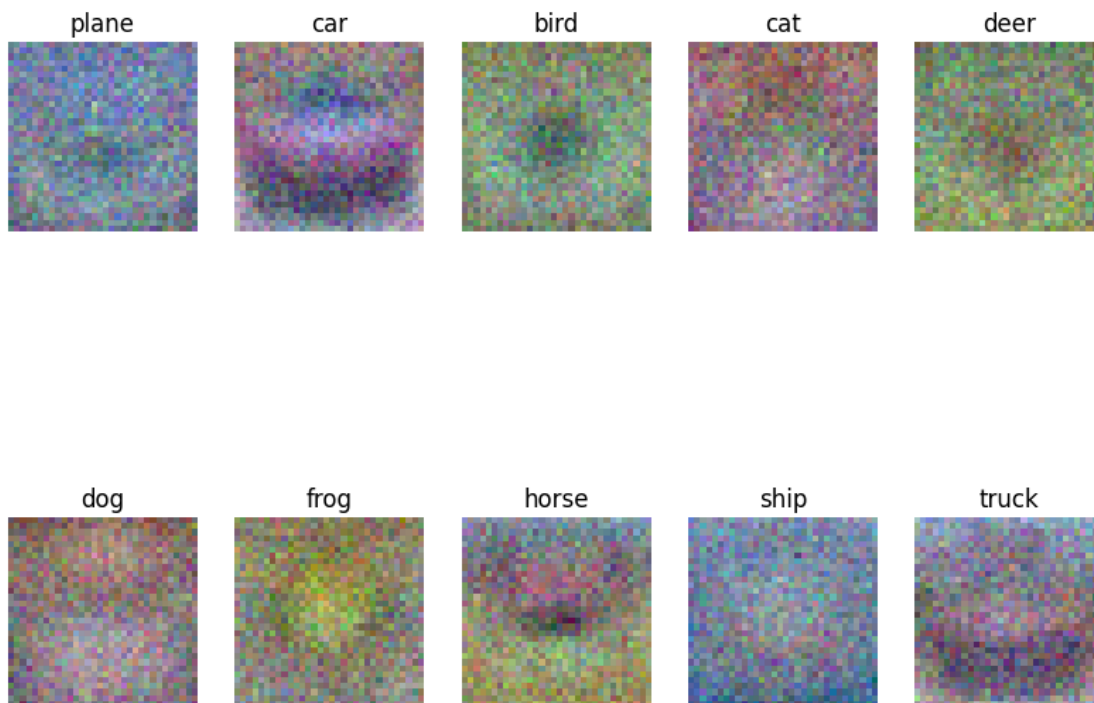
```
[20]: # Visualize the learned weights for each class.
# Depending on your choice of learning rate and regularization strength, these
# or may not be nice to look at.
```

```

w = best_softmax.W[:-1,:] # strip out the bias
w = w.reshape(32, 32, 3, 10)
w_min, w_max = np.min(w), np.max(w)
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship',
           'truck']
for i in range(10):
    plt.subplot(2, 5, i + 1)

    # Rescale the weights to be between 0 and 255
    wimg = 255.0 * (w[:, :, :, i].squeeze() - w_min) / (w_max - w_min)
    plt.imshow(wimg.astype('uint8'))
    plt.axis('off')
    plt.title(classes[i])

```



Inline question 3

Describe what your visualized Softmax classifier weights look like, and offer a brief explanation for why they look the way they do.

Your Answer: Each class weight image resembles a blurred prototype of the class. The plane and ship templates show blue regions for sky and water. The car and truck templates show blocky vehicle shapes. The horse template shows a body with legs and sometimes two heads, because the data mixes horses facing left and right. A linear classifier learns one averaged template per class, so each weight image looks like a smeared average of the training pictures.

[]:

q2_two_layer_net

Generative AI Use: For the purposes of the assignments, the use of generative AI is subject to the same policies regarding collaboration. Just as with other collaborators, each student must write down the solutions independently of the output of the interaction and the submission should include a note denoting the nature of the collaboration. The use of generative AI tools to substantially complete sections of the assignments is not in line with the spirit of the assignments, and would be a violation of the [Academic honesty, collaboration, AI tools usage](#) of this Course.

```
[1]: # Mounting your Assignment folder

# TODO: Enter the root path on your PC where you have saved the unzipped
# assignment folder, e.g. '/home/cliffbb/Documents/CSCD618-DSCD604/Assignments'
ROOT = r'D:\My Python Environment\Deep Learning And Neural Network_
↳Labs\Assignment'
# TODO: Enter the assignment folder e.g. 'assignment1'
ASSIGNMENT_FOLDER = 'Submit Files'

assert ROOT is not None, "[!] Enter the root path."
assert ASSIGNMENT_FOLDER is not None, "[!] Enter the assignment foldername."

import sys, os
from pathlib import Path
assignment_path = Path(os.path.join(ROOT, ASSIGNMENT_FOLDER))
sys.path.append(assignment_path)

# This downloads the CIFAR-10 dataset to your folder if it doesn't already_
↳exist.
%cd "$assignment_path"/csdc618_dscd604/datasets/
!bash get_datasets.sh
%cd "$assignment_path"
```

```
[WinError 3] The system cannot find the path specified: 'D:\My Python
Environment\Deep Learning And Neural Network Labs\Assignment\Submit Files
/cscd618_dscd604/datasets/'
D:\My Python Environment\Deep Learning And Neural Network Labs\Assignment\Submit
Files

D:\My Python Environment\Deep Learning And Neural Network Labs\Assignment\Submit
Files
```

bash: get_datasets.sh: No such file or directory

1 Fully-Connected Neural Nets

In this exercise we will implement fully-connected networks using a modular approach.

For each layer we will implement a **forward** and a **backward** function.

The **forward** function will receive inputs, weights, and other parameters and will return both an output and a **cache** object storing data needed for the backward pass, like this:

```
def layer_forward(x, w):
    """ Receive inputs x and weights w """
    # Do some computations ...
    z = # ... some intermediate value
    # Do some more computations ...
    out = # the output

    cache = (x, w, z, out) # Values we need to compute gradients

    return out, cache
```

The backward pass will receive upstream derivatives and the **cache** object, and will return gradients with respect to the inputs and weights, like this:

```
def layer_backward(dout, cache):
    """
    Receive dout (derivative of loss with respect to outputs) and cache,
    and compute derivative with respect to inputs.
    """
    # Unpack cache values
    x, w, z, out = cache

    # Use values in cache to compute derivatives
    dx = # Derivative of loss with respect to x
    dw = # Derivative of loss with respect to w

    return dx, dw
```

After implementing a bunch of layers this way, we will be able to easily combine them to build classifiers with different architectures.

```
[2]: # As usual, a bit of setup
from __future__ import print_function
import time
import numpy as np
import matplotlib.pyplot as plt
from csdc618_dscd604.classifiers.fc_net import *
from csdc618_dscd604.data_utils import get_CIFAR10_data
```

```

from csdc618_dscd604.gradient_check import eval_numerical_gradient, \
    eval_numerical_gradient_array
from csdc618_dscd604.solver import Solver

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading external modules
# see http://stackoverflow.com/questions/1907993/
# autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2

def rel_error(x, y):
    """ returns relative error """
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))

```

[3]: # Load the (preprocessed) CIFAR10 data.

```

data = get_CIFAR10_data()
for k, v in list(data.items()):
    print('%s: ' % k, v.shape)

('X_train: ', (49000, 3, 32, 32))
('y_train: ', (49000,))
('X_val: ', (1000, 3, 32, 32))
('y_val: ', (1000,))
('X_test: ', (1000, 3, 32, 32))
('y_test: ', (1000,))

```

2 Affine layer: forward

Open the file `csdc618_dscd604/layers.py` and implement the `affine_forward` function.

Once you are done you can test your implementation by running the following:

[4]: # Test the `affine_forward` function

```

num_inputs = 2
input_shape = (4, 5, 6)
output_dim = 3

input_size = num_inputs * np.prod(input_shape)
weight_size = output_dim * np.prod(input_shape)

x = np.linspace(-0.1, 0.5, num=input_size).reshape(num_inputs, *input_shape)

```

```

w = np.linspace(-0.2, 0.3, num=weight_size).reshape(np.prod(input_shape),  

↳output_dim)
b = np.linspace(-0.3, 0.1, num=output_dim)

out, _ = affine_forward(x, w, b)
correct_out = np.array([[ 1.49834967,  1.70660132,  1.91485297],
                        [ 3.25553199,  3.5141327,  3.77273342]])

# Compare your output with ours. The error should be around e-9 or less.
print('Testing affine_forward function:')
print('difference: ', rel_error(out, correct_out))

```

Testing affine_forward function:
difference: 9.769848888397517e-10

3 Affine layer: backward

Now implement the `affine_backward` function in the file `csdc618_dscd604/layers.py` and test your implementation using numeric gradient checking.

```

[5]: # Test the affine_backward function
np.random.seed(231)
x = np.random.randn(10, 2, 3)
w = np.random.randn(6, 5)
b = np.random.randn(5)
dout = np.random.randn(10, 5)

dx_num = eval_numerical_gradient_array(lambda x: affine_forward(x, w, b)[0], x,  

↳dout)
dw_num = eval_numerical_gradient_array(lambda w: affine_forward(x, w, b)[0], w,  

↳dout)
db_num = eval_numerical_gradient_array(lambda b: affine_forward(x, w, b)[0], b,  

↳dout)

_, cache = affine_forward(x, w, b)
dx, dw, db = affine_backward(dout, cache)

# The error should be around e-10 or less
print('Testing affine_backward function:')
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))

```

Testing affine_backward function:
dx error: 5.399100368651805e-11
dw error: 9.904211865398145e-11
db error: 2.4122867568119087e-11

4 ReLU activation: forward

Implement the forward pass `relu_forward` for the ReLU activation function in the file `csdc618_dsdc604/layers.py` and test your implementation using the following:

```
[6]: # Test the relu_forward function

x = np.linspace(-0.5, 0.5, num=12).reshape(3, 4)

out, _ = relu_forward(x)
correct_out = np.array([[ 0.,          0.,          0.,          0.,          ],
                        [ 0.,          0.,          0.04545455, 0.13636364,],
                        [ 0.22727273, 0.31818182, 0.40909091, 0.5,          ]])

# Compare your output with ours. The error should be on the order of e-8
print('Testing relu_forward function:')
print('difference: ', rel_error(out, correct_out))
```

```
Testing relu_forward function:
difference: 4.999999798022158e-08
```

5 ReLU activation: backward

Now implement the backward pass `relu_backward` for the ReLU activation function in the file `csdc618_dsdc604/layers.py` and test your implementation using numeric gradient checking:

```
[7]: np.random.seed(231)
x = np.random.randn(10, 10)
dout = np.random.randn(*x.shape)

dx_num = eval_numerical_gradient_array(lambda x: relu_forward(x)[0], x, dout)

_, cache = relu_forward(x)
dx = relu_backward(dout, cache)

# The error should be on the order of e-12
print('Testing relu_backward function:')
print('dx error: ', rel_error(dx_num, dx))
```

```
Testing relu_backward function:
dx error: 3.2756349136310288e-12
```

5.1 Inline Question 1:

We've only asked you to implement ReLU, but there are a number of different activation functions that one could use in neural networks, each with its pros and cons. In particular, an issue commonly seen with activation functions is getting zero (or close to zero) gradient flow during backpropagation. Which of the following activation functions have this problem? If you consider these functions in

the one dimensional case, what types of input would lead to this behaviour? 1. Sigmoid 2. ReLU 3. Leaky ReLU

Your Answer: Sigmoid and ReLU show this problem. Sigmoid saturates for large positive or large negative input, so the gradient at both tails approaches zero. ReLU outputs zero for negative input, so the gradient is zero for any negative input and a unit stuck there stops learning. Leaky ReLU keeps a small positive slope for negative input, so the gradient stays nonzero and avoids the problem.

6 “Sandwich” layers

There are some common patterns of layers that are frequently used in neural nets.

For example, affine layers are frequently followed by a ReLU nonlinearity.

To make these common patterns easy, we define several convenience layers in the file `csdc618_dscd604/layer_utils.py`.

For now take a look at the `affine_relu_forward` and `affine_relu_backward` functions, and run the following to numerically gradient check the backward pass:

```
[8]: from csdc618_dscd604.layer_utils import affine_relu_forward, \
      ↪ affine_relu_backward
np.random.seed(231)
x = np.random.randn(2, 3, 4)
w = np.random.randn(12, 10)
b = np.random.randn(10)
dout = np.random.randn(2, 10)

out, cache = affine_relu_forward(x, w, b)
dx, dw, db = affine_relu_backward(dout, cache)

dx_num = eval_numerical_gradient_array(lambda x: affine_relu_forward(x, w, \
      ↪ b)[0], x, dout)
dw_num = eval_numerical_gradient_array(lambda w: affine_relu_forward(x, w, \
      ↪ b)[0], w, dout)
db_num = eval_numerical_gradient_array(lambda b: affine_relu_forward(x, w, \
      ↪ b)[0], b, dout)

# Relative error should be around e-10 or less
print('Testing affine_relu_forward and affine_relu_backward:')
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))
```

Testing affine_relu_forward and affine_relu_backward:

dx error: 2.299579177309368e-11

dw error: 8.162011105764925e-11

db error: 7.826724021458994e-12

7 Loss layers: Softmax

Now implement the loss and gradient for softmax in the `softmax_loss` function in `csdc618_dscd604/layers.py`.

This should be similar to what you implemented in `csdc618_dscd604/classifiers/softmax.py`.

You can make sure that the implementations are correct by running the following:

```
[9]: np.random.seed(231)
num_classes, num_inputs = 10, 50
x = 0.001 * np.random.randn(num_inputs, num_classes)
y = np.random.randint(num_classes, size=num_inputs)

dx_num = eval_numerical_gradient(lambda x: softmax_loss(x, y)[0], x,
    verbose=False)
loss, dx = softmax_loss(x, y)

# Test softmax_loss function. Loss should be close to 2.3 and dx error should
    be around e-8
print('\nTesting softmax_loss:')
print('loss: ', loss)
print('dx error: ', rel_error(dx_num, dx))
```

Testing softmax_loss:

loss: 2.302545844500738

dx error: 9.384673161989355e-09

8 Two-layer network

Open the file `csdc618_dscd604/classifiers/fc_net.py` and complete the implementation of the `TwoLayerNet` class.

Read through it to make sure you understand the API. You can run the cell below to test your implementation.

```
[10]: np.random.seed(231)
N, D, H, C = 3, 5, 50, 7
X = np.random.randn(N, D)
y = np.random.randint(C, size=N)

std = 1e-3
model = TwoLayerNet(input_dim=D, hidden_dim=H, num_classes=C, weight_scale=std)

print('Testing initialization ... ')
W1_std = abs(model.params['W1'].std() - std)
b1 = model.params['b1']
W2_std = abs(model.params['W2'].std() - std)
```

```

b2 = model.params['b2']
assert W1_std < std / 10, 'First layer weights do not seem right'
assert np.all(b1 == 0), 'First layer biases do not seem right'
assert W2_std < std / 10, 'Second layer weights do not seem right'
assert np.all(b2 == 0), 'Second layer biases do not seem right'

print('Testing test-time forward pass ... ')
model.params['W1'] = np.linspace(-0.7, 0.3, num=D*H).reshape(D, H)
model.params['b1'] = np.linspace(-0.1, 0.9, num=H)
model.params['W2'] = np.linspace(-0.3, 0.4, num=H*C).reshape(H, C)
model.params['b2'] = np.linspace(-0.9, 0.1, num=C)
X = np.linspace(-5.5, 4.5, num=N*D).reshape(D, N).T
scores = model.loss(X)
correct_scores = np.asarray(
    [[11.53165108, 12.2917344, 13.05181771, 13.81190102, 14.57198434, 15.
    ↪33206765, 16.09215096],
    [12.05769098, 12.74614105, 13.43459113, 14.1230412, 14.81149128, 15.
    ↪49994135, 16.18839143],
    [12.58373087, 13.20054771, 13.81736455, 14.43418138, 15.05099822, 15.
    ↪66781506, 16.2846319 ]])
scores_diff = np.abs(scores - correct_scores).sum()
assert scores_diff < 1e-6, 'Problem with test-time forward pass'

print('Testing training loss (no regularization)')
y = np.asarray([0, 5, 1])
loss, grads = model.loss(X, y)
correct_loss = 3.4702243556
assert abs(loss - correct_loss) < 1e-10, 'Problem with training-time loss'

model.reg = 1.0
loss, grads = model.loss(X, y)
correct_loss = 26.5948426952
assert abs(loss - correct_loss) < 1e-10, 'Problem with regularization loss'

# Errors should be around e-7 or less
for reg in [0.0, 0.7]:
    print('Running numeric gradient check with reg = ', reg)
    model.reg = reg
    loss, grads = model.loss(X, y)

    for name in sorted(grads):
        f = lambda _: model.loss(X, y)[0]
        grad_num = eval_numerical_gradient(f, model.params[name], verbose=False)
        print('%s relative error: %.2e' % (name, rel_error(grad_num, grads[name])))

```

Testing initialization ...


```

Testing test-time forward pass ...
Testing training loss (no regularization)
Running numeric gradient check with reg = 0.0
W1 relative error: 1.52e-08

W2 relative error: 3.21e-10
b1 relative error: 8.37e-09
b2 relative error: 4.33e-10
Running numeric gradient check with reg = 0.7
W1 relative error: 2.53e-07

W2 relative error: 2.85e-08
b1 relative error: 1.56e-08
b2 relative error: 7.76e-10

```

9 Solver

Open the file `csdc618_dscd604/solver.py` and read through it to familiarize yourself with the API.

After doing so, use a `Solver` instance to train a `TwoLayerNet` that achieves about 36% accuracy on the validation set.

```

[11]: input_size = 32 * 32 * 3
hidden_size = 50
num_classes = 10
model = TwoLayerNet(input_size, hidden_size, num_classes)
solver = None

#####
# TODO: Use a Solver instance to train a TwoLayerNet that achieves about 36% #
# accuracy on the validation set.                                           #
#####
# *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
solver = Solver(
    model,
    data,
    update_rule='sgd',
    optim_config={'learning_rate': 1e-3},
    lr_decay=0.95,
    num_epochs=15,
    batch_size=200,
    print_every=400,
    verbose=True,
)
solver.train()
# *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
#####
#
#                                     END OF YOUR CODE
#

```

```
#####
```

```
(Iteration 1 / 3675) loss: 2.301725
(Epoch 0 / 15) train acc: 0.187000; val_acc: 0.181000
(Epoch 1 / 15) train acc: 0.383000; val_acc: 0.418000
(Iteration 401 / 3675) loss: 1.543941
(Epoch 2 / 15) train acc: 0.444000; val_acc: 0.439000
(Epoch 3 / 15) train acc: 0.476000; val_acc: 0.444000
(Iteration 801 / 3675) loss: 1.515084
(Epoch 4 / 15) train acc: 0.495000; val_acc: 0.469000
(Iteration 1201 / 3675) loss: 1.612220
(Epoch 5 / 15) train acc: 0.509000; val_acc: 0.477000
(Epoch 6 / 15) train acc: 0.535000; val_acc: 0.480000
(Iteration 1601 / 3675) loss: 1.405752
(Epoch 7 / 15) train acc: 0.543000; val_acc: 0.508000
(Epoch 8 / 15) train acc: 0.524000; val_acc: 0.488000
(Iteration 2001 / 3675) loss: 1.294265
(Epoch 9 / 15) train acc: 0.575000; val_acc: 0.509000
(Iteration 2401 / 3675) loss: 1.320206
(Epoch 10 / 15) train acc: 0.528000; val_acc: 0.504000
(Epoch 11 / 15) train acc: 0.552000; val_acc: 0.500000
(Iteration 2801 / 3675) loss: 1.290650
(Epoch 12 / 15) train acc: 0.566000; val_acc: 0.512000
(Epoch 13 / 15) train acc: 0.561000; val_acc: 0.495000
(Iteration 3201 / 3675) loss: 1.350225
(Epoch 14 / 15) train acc: 0.589000; val_acc: 0.511000
(Iteration 3601 / 3675) loss: 1.171868
(Epoch 15 / 15) train acc: 0.567000; val_acc: 0.525000
```

10 Debug the training

With the default parameters we provided above, you should get a validation accuracy of about 0.36 on the validation set. This isn't very good.

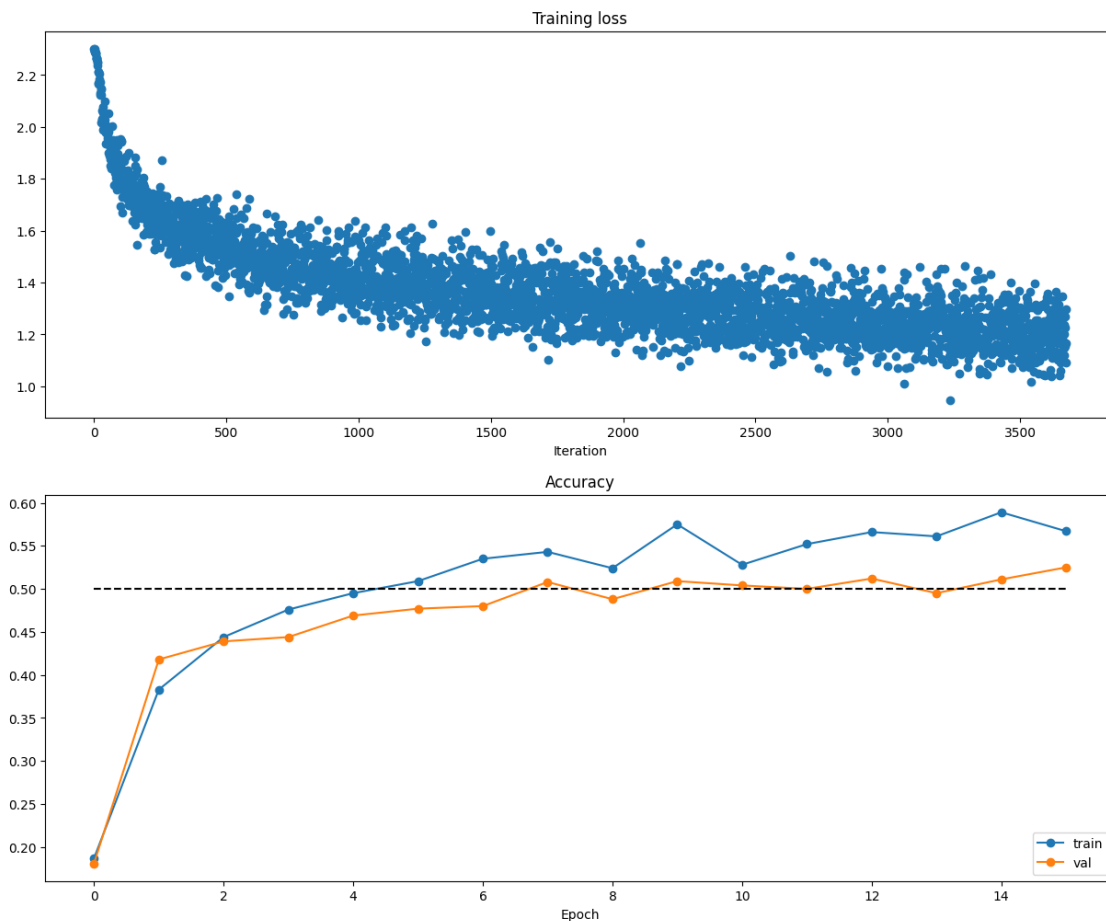
One strategy for getting insight into what's wrong is to plot the loss function and the accuracies on the training and validation sets during optimization.

Another strategy is to visualize the weights that were learned in the first layer of the network. In most neural networks trained on visual data, the first layer weights typically show some visible structure when visualized.

```
[12]: # Run this cell to visualize training loss and train / val accuracy
```

```
plt.subplot(2, 1, 1)
plt.title('Training loss')
plt.plot(solver.loss_history, 'o')
plt.xlabel('Iteration')

plt.subplot(2, 1, 2)
plt.title('Accuracy')
plt.plot(solver.train_acc_history, '-o', label='train')
plt.plot(solver.val_acc_history, '-o', label='val')
plt.plot([0.5] * len(solver.val_acc_history), 'k--')
plt.xlabel('Epoch')
plt.legend(loc='lower right')
plt.gcf().set_size_inches(15, 12)
plt.show()
```

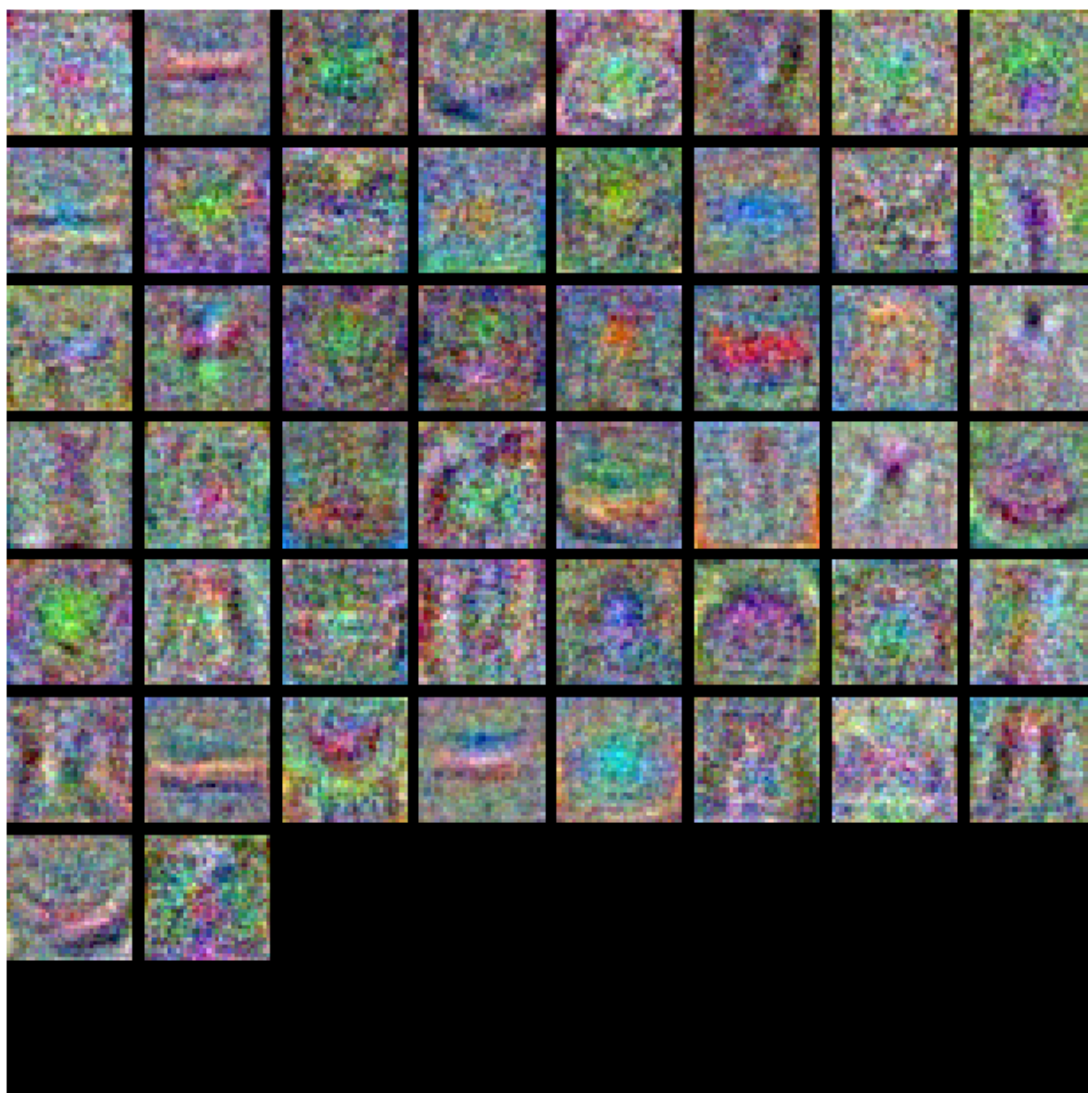


```
[13]: from csdc618_dscd604.vis_utils import visualize_grid

# Visualize the weights of the network

def show_net_weights(net):
    W1 = net.params['W1']
    W1 = W1.reshape(3, 32, 32, -1).transpose(3, 1, 2, 0)
    plt.imshow(visualize_grid(W1, padding=3).astype('uint8'))
    plt.gca().axis('off')
    plt.show()

show_net_weights(model)
```



11 Tune your hyperparameters

What's wrong?. Looking at the visualizations above, we see that the loss is decreasing more or less linearly, which seems to suggest that the learning rate may be too low. Moreover, there is no gap between the training and validation accuracy, suggesting that the model we used has low capacity, and that we should increase its size. On the other hand, with a very large model we would expect to see more overfitting, which would manifest itself as a very large gap between the training and validation accuracy.

Tuning. Tuning the hyperparameters and developing intuition for how they affect the final performance is a large part of using Neural Networks, so we want you to get a lot of practice. Below, you should experiment with different values of the various hyperparameters, including hidden layer size, learning rate, number of training epochs, and regularization strength. You might also consider tuning the learning rate decay, but you should be able to get good performance using the default value.

Approximate results. You should be aim to achieve a classification accuracy of greater than 48% on the validation set. Our best network gets over 52% on the validation set.

Experiment: Your goal in this exercise is to get as good of a result on CIFAR-10 as you can (52% could serve as a reference), with a fully-connected Neural Network. Feel free implement your own techniques (e.g. PCA to reduce dimensionality, or adding dropout, or adding features to the solver, etc.).

```
[14]: best_model = None

#####
# TODO: Tune hyperparameters using the validation set. Store your best trained
#
# model in best_model.
#
#
# To help debug your network, it may help to use visualizations similar to the
# ones we used above; these visualizations will have significant qualitative
# differences from the ones we saw above for the poorly tuned network.
#
#
# Tweaking hyperparameters by hand can be fun, but you might find it useful to
# write code to sweep through possible combinations of hyperparameters
```

```

# automatically like we did on thes previous exercises.
↪#
#####
# *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
input_size = 32 * 32 * 3
num_classes = 10
best_val = -1
for hidden_size in [128]:
    for lr in [1e-3]:
        for reg in [0.0]:
            model = TwoLayerNet(input_size, hidden_size, num_classes, reg=reg)
            solver = Solver(
                model, data,
                update_rule='sgd',
                optim_config={'learning_rate': lr},
                lr_decay=0.95, num_epochs=20, batch_size=200,
                verbose=False, print_every=100000,
            )
            solver.train()
            print('hidden %d lr %.0e reg %.0e -> val %.4f'
                  % (hidden_size, lr, reg, solver.best_val_acc))
            if solver.best_val_acc > best_val:
                best_val = solver.best_val_acc
                best_model = model
print('best validation accuracy:', best_val)
# *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
#####
#                                     END OF YOUR CODE
↪#
#####

```

```

hidden 128 lr 1e-03 reg 0e+00 -> val 0.5350
best validation accuracy: 0.535

```

12 Test your model!

Run your best model on the validation and test sets. You should achieve above 48% accuracy on the validation set and the test set.

```

[15]: y_val_pred = np.argmax(best_model.loss(data['X_val']), axis=1)
      print('Validation set accuracy: ', (y_val_pred == data['y_val']).mean())

```

```
Validation set accuracy: 0.535
```

```

[16]: y_test_pred = np.argmax(best_model.loss(data['X_test']), axis=1)
      print('Test set accuracy: ', (y_test_pred == data['y_test']).mean())

```

```
Test set accuracy: 0.525
```

```
[17]: # Save best model
best_model.save("best_two_layer_net.npy")
```

best_two_layer_net.npy saved.

12.1 Inline Question 2:

Now that you have trained a Neural Network classifier, you may find that your testing accuracy is much lower than the training accuracy. In what ways can we decrease this gap? Select all that apply.

1. Train on a larger dataset.
2. Add more hidden units.
3. Increase the regularization strength.
4. None of the above.

Your Answer: 1 and 3.

Your Explanation: A larger training set exposes the model to more variation, so the model generalizes better and the gap shrinks. Stronger regularization lowers training accuracy and raises validation accuracy, so the gap shrinks. Adding hidden units raises capacity and fits the training data more tightly, so the gap grows. Option 2 widens the gap rather than shrinking it.

[]:

q3_features

Generative AI Use: For the purposes of the assignments, the use of generative AI is subject to the same policies regarding collaboration. Just as with other collaborators, each student must write down the solutions independently of the output of the interaction and the submission should include a note denoting the nature of the collaboration. The use of generative AI tools to substantially complete sections of the assignments is not in line with the spirit of the assignments, and would be a violation of the [Academic honesty, collaboration, AI tools usage](#) of this Course.

```
[1]: # Mounting your Assignment folder

# TODO: Enter the root path on your PC where you have saved the unzipped
# assignment folder, e.g. '/home/cliffbb/Documents/CSCD618-DSCD604/Assignments'
ROOT = r'D:\My Python Environment\Deep Learning And Neural Network\
↳Labs\Assignment'
# TODO: Enter the assignment folder e.g. 'assignment1'
ASSIGNMENT_FOLDER = 'Submit Files'

assert ROOT is not None, "[!] Enter the root path."
assert ASSIGNMENT_FOLDER is not None, "[!] Enter the assignment foldername."

import sys, os
from pathlib import Path
assignment_path = Path(os.path.join(ROOT, ASSIGNMENT_FOLDER))
sys.path.append(assignment_path)

# This downloads the CIFAR-10 dataset to your folder if it doesn't already
↳exist.
%cd "$assignment_path"/csdc618_dscd604/datasets/
!bash get_datasets.sh
%cd "$assignment_path"
```

```
[WinError 3] The system cannot find the path specified: 'D:\\My Python
Environment\\Deep Learning And Neural Network Labs\\Assignment\\Submit Files
/cscd618_dscd604/datasets/'
```

```
D:\My Python Environment\Deep Learning And Neural Network Labs\Assignment\Submit
Files
```

```
D:\My Python Environment\Deep Learning And Neural Network Labs\Assignment\Submit
Files
```


bash: get_datasets.sh: No such file or directory

1 Image Features Exercise

We have seen that we can achieve reasonable performance on an image classification task by training a linear classifier on the pixels of the input image.

In this exercise we will show that we can improve our classification performance by training linear classifiers not on raw pixels but on features that are computed from the raw pixels.

All of your work for this exercise will be done in this notebook.

```
[2]: import random
import numpy as np
from csdc618_dscd604.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading external modules
# see http://stackoverflow.com/questions/1907993/
#   ↳ autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2
```

1.1 Load data

Similar to previous exercises, we will load CIFAR-10 data from disk.

```
[3]: from csdc618_dscd604.features import color_histogram_hsv, hog_feature

def get_CIFAR10_data(num_training=49000, num_validation=1000, num_test=1000):
    # Load the raw CIFAR-10 data
    cifar10_dir = 'csdc618_dscd604/datasets/cifar-10-batches-py'

    # Cleaning up variables to prevent loading data multiple times (which may
    #   ↳ cause memory issue)
    try:
        del X_train, y_train
        del X_test, y_test
        print('Clear previously loaded data.')
    except:
        pass
```

```

X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

# Subsample the data
mask = list(range(num_training, num_training + num_validation))
X_val = X_train[mask]
y_val = y_train[mask]
mask = list(range(num_training))
X_train = X_train[mask]
y_train = y_train[mask]
mask = list(range(num_test))
X_test = X_test[mask]
y_test = y_test[mask]

return X_train, y_train, X_val, y_val, X_test, y_test

X_train, y_train, X_val, y_val, X_test, y_test = get_CIFAR10_data()

```

1.2 Extract Features

For each image we will compute a Histogram of Oriented Gradients (HOG) as well as a color histogram using the hue channel in HSV color space. We form our final feature vector for each image by concatenating the HOG and color histogram feature vectors.

Roughly speaking, HOG should capture the texture of the image while ignoring color information, and the color histogram represents the color of the input image while ignoring texture. As a result, we expect that using both together ought to work better than using either alone. Verifying this assumption would be a good thing to try for your own interest.

The `hog_feature` and `color_histogram_hsv` functions both operate on a single image and return a feature vector for that image. The `extract_features` function takes a set of images and a list of feature functions and evaluates each feature function on each image, storing the results in a matrix where each column is the concatenation of all feature vectors for a single image.

```

[4]: from csdc618_dscd604.features import *

# num_color_bins = 10 # Number of bins in the color histogram
num_color_bins = 25 # Number of bins in the color histogram
feature_fns = [hog_feature, lambda img: color_histogram_hsv(img,
↳nbin=num_color_bins)]
X_train_feats = extract_features(X_train, feature_fns, verbose=True)
X_val_feats = extract_features(X_val, feature_fns)
X_test_feats = extract_features(X_test, feature_fns)

# Preprocessing: Subtract the mean feature
mean_feat = np.mean(X_train_feats, axis=0, keepdims=True)
X_train_feats -= mean_feat
X_val_feats -= mean_feat
X_test_feats -= mean_feat

```

```

# Preprocessing: Divide by standard deviation. This ensures that each feature
# has roughly the same scale.
std_feat = np.std(X_train_feats, axis=0, keepdims=True)
X_train_feats /= std_feat
X_val_feats /= std_feat
X_test_feats /= std_feat

# Preprocessing: Add a bias dimension
X_train_feats = np.hstack([X_train_feats, np.ones((X_train_feats.shape[0], 1))])
X_val_feats = np.hstack([X_val_feats, np.ones((X_val_feats.shape[0], 1))])
X_test_feats = np.hstack([X_test_feats, np.ones((X_test_feats.shape[0], 1))])

```

```

Done extracting features for 1000 / 49000 images
Done extracting features for 2000 / 49000 images
Done extracting features for 3000 / 49000 images
Done extracting features for 4000 / 49000 images
Done extracting features for 5000 / 49000 images
Done extracting features for 6000 / 49000 images
Done extracting features for 7000 / 49000 images
Done extracting features for 8000 / 49000 images
Done extracting features for 9000 / 49000 images
Done extracting features for 10000 / 49000 images
Done extracting features for 11000 / 49000 images
Done extracting features for 12000 / 49000 images
Done extracting features for 13000 / 49000 images
Done extracting features for 14000 / 49000 images
Done extracting features for 15000 / 49000 images
Done extracting features for 16000 / 49000 images
Done extracting features for 17000 / 49000 images
Done extracting features for 18000 / 49000 images
Done extracting features for 19000 / 49000 images
Done extracting features for 20000 / 49000 images
Done extracting features for 21000 / 49000 images
Done extracting features for 22000 / 49000 images
Done extracting features for 23000 / 49000 images

```

Done extracting features for 24000 / 49000 images
Done extracting features for 25000 / 49000 images
Done extracting features for 26000 / 49000 images
Done extracting features for 27000 / 49000 images
Done extracting features for 28000 / 49000 images
Done extracting features for 29000 / 49000 images
Done extracting features for 30000 / 49000 images
Done extracting features for 31000 / 49000 images
Done extracting features for 32000 / 49000 images
Done extracting features for 33000 / 49000 images
Done extracting features for 34000 / 49000 images
Done extracting features for 35000 / 49000 images
Done extracting features for 36000 / 49000 images
Done extracting features for 37000 / 49000 images
Done extracting features for 38000 / 49000 images
Done extracting features for 39000 / 49000 images
Done extracting features for 40000 / 49000 images
Done extracting features for 41000 / 49000 images
Done extracting features for 42000 / 49000 images
Done extracting features for 43000 / 49000 images
Done extracting features for 44000 / 49000 images
Done extracting features for 45000 / 49000 images
Done extracting features for 46000 / 49000 images
Done extracting features for 47000 / 49000 images
Done extracting features for 48000 / 49000 images
Done extracting features for 49000 / 49000 images

1.3 Train Softmax classifier on features

Using the `Softmax` code developed earlier in the assignment, train `Softmax` classifiers on top of the features extracted above; this should achieve better results than training them directly on top of raw pixels.

[5]: *# Use the validation set to tune the learning rate and regularization strength*

```

from csdc618_dscd604.classifiers.linear_classifier import Softmax

learning_rates = [1e-7, 1e-6]
regularization_strengths = [5e5, 5e6]

results = {}
best_val = -1
best_softmax = None

#####
# TODO:
# Use the validation set to set the learning rate and regularization strength. #
# This should be identical to the validation that you did for the Softmax; save#
# the best trained classifier in best_softmax. If you carefully tune the model, #
# you should be able to get accuracy of above 0.42 on the validation set.      #
#####
# *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
learning_rates = [1e-7, 3e-7, 1e-6]
regularization_strengths = [1e4, 5e4, 1e5]
for lr in learning_rates:
    for reg in regularization_strengths:
        model = Softmax()
        model.train(X_train_feats, y_train, learning_rate=lr, reg=reg,
                    num_iters=3000, verbose=False)
        train_accuracy = np.mean(y_train == model.predict(X_train_feats))
        val_accuracy = np.mean(y_val == model.predict(X_val_feats))
        results[(lr, reg)] = (train_accuracy, val_accuracy)
        if val_accuracy > best_val:
            best_val = val_accuracy
            best_softmax = model
# *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
#####
#                                     END OF YOUR CODE                                     #
#####

# Print out results.
for lr, reg in sorted(results):
    train_accuracy, val_accuracy = results[(lr, reg)]
    print('lr %e reg %e train accuracy: %f val accuracy: %f' % (
        lr, reg, train_accuracy, val_accuracy))

print('best validation accuracy achieved: %f' % best_val)

```

```

lr 1.000000e-07 reg 1.000000e+04 train accuracy: 0.311102 val accuracy: 0.303000
lr 1.000000e-07 reg 5.000000e+04 train accuracy: 0.420061 val accuracy: 0.416000
lr 1.000000e-07 reg 1.000000e+05 train accuracy: 0.424878 val accuracy: 0.441000
lr 3.000000e-07 reg 1.000000e+04 train accuracy: 0.421469 val accuracy: 0.428000
lr 3.000000e-07 reg 5.000000e+04 train accuracy: 0.423082 val accuracy: 0.428000

```

```

lr 3.000000e-07 reg 1.000000e+05 train accuracy: 0.416551 val accuracy: 0.428000
lr 1.000000e-06 reg 1.000000e+04 train accuracy: 0.417837 val accuracy: 0.416000
lr 1.000000e-06 reg 5.000000e+04 train accuracy: 0.417653 val accuracy: 0.422000
lr 1.000000e-06 reg 1.000000e+05 train accuracy: 0.398388 val accuracy: 0.404000
best validation accuracy achieved: 0.441000

```

```

[6]: # Evaluate your trained Softmax on the test set: you should be able to get at least 0.42
y_test_pred = best_softmax.predict(X_test_feats)
test_accuracy = np.mean(y_test == y_test_pred)
print(test_accuracy)

```

0.44

```

[7]: # Save best softmax model
best_softmax.save("best_softmax_features.npy")

```

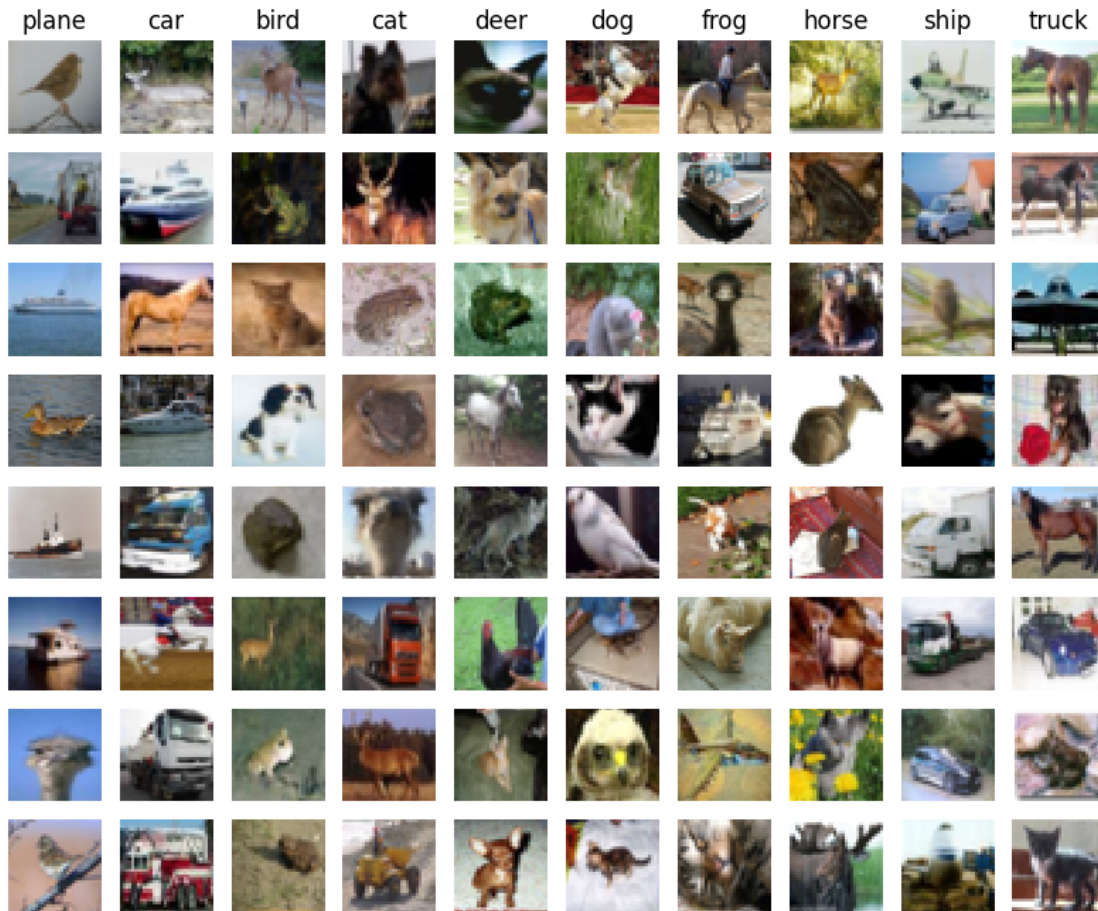
best_softmax_features.npy saved.

```

[8]: # An important way to gain intuition about how an algorithm works is to
# visualize the mistakes that it makes. In this visualization, we show examples
# of images that are misclassified by our current system. The first column
# shows images that our system labeled as "plane" but whose true label is
# something other than "plane".

examples_per_class = 8
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',
           'ship', 'truck']
for cls, cls_name in enumerate(classes):
    idxs = np.where((y_test != cls) & (y_test_pred == cls))[0]
    idxs = np.random.choice(idxs, examples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt.subplot(examples_per_class, len(classes), i * len(classes) + cls + 1)
        plt.imshow(X_test[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls_name)
plt.show()

```



1.3.1 Inline question 1:

Describe the misclassification results that you see. Do they make sense?

Your Answer: Yes, the mistakes make sense. The wrong images share color or texture with the predicted class. Planes mix with ship and bird because of blue and gray backgrounds. Cats and dogs swap often because of similar fur texture and body shape. HOG and the hue histogram capture coarse texture and color, not fine object parts, so visually similar classes collide.

1.4 Neural Network on image features

Earlier in this assignment we saw that training a two-layer neural network on raw pixels achieved better classification performance than linear classifiers on raw pixels.

In this notebook we have seen that linear classifiers on image features outperform linear classifiers on raw pixels.

For completeness, we should also try training a neural network on image features.

This approach should outperform all previous approaches: you should easily be able to achieve

over 55% classification accuracy on the test set; our best model achieves about 60% classification accuracy.

```
[9]: # Preprocessing: Remove the bias dimension
# Make sure to run this cell only ONCE
print(X_train_feats.shape)
X_train_feats = X_train_feats[:, :-1]
X_val_feats = X_val_feats[:, :-1]
X_test_feats = X_test_feats[:, :-1]

print(X_train_feats.shape)
```

(49000, 170)

(49000, 169)

```
[10]: from csdc618_dsdc604.classifiers.fc_net import TwoLayerNet
from csdc618_dsdc604.solver import Solver

input_dim = X_train_feats.shape[1]
hidden_dim = 500
num_classes = 10

data = {
    'X_train': X_train_feats,
    'y_train': y_train,
    'X_val': X_val_feats,
    'y_val': y_val,
    'X_test': X_test_feats,
    'y_test': y_test,
}

net = TwoLayerNet(input_dim, hidden_dim, num_classes)
best_net = None

#####
# TODO: Train a two-layer neural network on image features. You may want to #
# cross-validate various parameters as in previous sections. Store your best #
# model in the best_net variable. #
#####
# *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
best_val = -1
for lr in [0.1, 0.2]:
    for reg in [1e-3]:
        net = TwoLayerNet(input_dim, hidden_dim, num_classes, reg=reg)
        solver = Solver(
            net, data,
            update_rule='sgd_momentum',
            optim_config={'learning_rate': lr},
```



```

        lr_decay=0.95, num_epochs=25, batch_size=200,
        verbose=False, print_every=100000,
    )
    solver.train()
    print('lr %.2f reg %.0e -> val %.4f' % (lr, reg, solver.best_val_acc))
    if solver.best_val_acc > best_val:
        best_val = solver.best_val_acc
        best_net = net
    print('best validation accuracy:', best_val)
    # *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
    #####
    #                                     END OF YOUR CODE                                     #
    #####

```

lr 0.10 reg 1e-03 -> val 0.5990

lr 0.20 reg 1e-03 -> val 0.5940

best validation accuracy: 0.599

[11]: *# Run your best neural net classifier on the test set. You should be able
to get more than 58% accuracy. It is also possible to get >60% accuracy
with careful tuning.*

```

y_test_pred = np.argmax(best_net.loss(data['X_test']), axis=1)
test_acc = (y_test_pred == data['y_test']).mean()
print(test_acc)

```

0.586

[12]: *# Save best model*
best_net.save("best_two_layer_net_features.npy")

best_two_layer_net_features.npy saved.

[]:

q4_fully_connected_nets

Generative AI Use: For the purposes of the assignments, the use of generative AI is subject to the same policies regarding collaboration. Just as with other collaborators, each student must write down the solutions independently of the output of the interaction and the submission should include a note denoting the nature of the collaboration. The use of generative AI tools to substantially complete sections of the assignments is not in line with the spirit of the assignments, and would be a violation of the [Academic honesty, collaboration, AI tools usage](#) of thi Course.

```
[1]: # Mounting your Assignment folder

# TODO: Enter the root path on your PC where you have saved the unzipped
# assignment folder, e.g. '/home/cliffbb/Documents/CSCD618-DSCD604/Assignments'
ROOT = r'D:\My Python Environment\Deep Learning And Neural Network\
↳Labs\Assignment'
# TODO: Enter the assignment folder e.g. 'assignment1'
ASSIGNMENT_FOLDER = 'Submit Files'

assert ROOT is not None, "[!] Enter the root path."
assert ASSIGNMENT_FOLDER is not None, "[!] Enter the assignment foldername."

import sys, os
from pathlib import Path
assignment_path = Path(os.path.join(ROOT, ASSIGNMENT_FOLDER))
sys.path.append(assignment_path)

# This downloads the CIFAR-10 dataset to your folder if it doesn't already
↳exist.
%cd "$assignment_path"/csdc618_dscd604/datasets/
!bash get_datasets.sh
%cd "$assignment_path"
```

```
[WinError 3] The system cannot find the path specified: 'D:\\My Python
Environment\\Deep Learning And Neural Network Labs\\Assignment\\Submit Files
/cscd618_dscd604/datasets/'
D:\My Python Environment\Deep Learning And Neural Network Labs\Assignment\Submit
Files

D:\My Python Environment\Deep Learning And Neural Network Labs\Assignment\Submit
Files
```

bash: get_datasets.sh: No such file or directory

1 Multi-Layer Fully Connected Network

In this exercise, you will implement a **fully connected network** with an arbitrary number of hidden layers.

Read through the FullyConnectedNet class in the file csdc618_dscd604/classifiers/fc_net.py.

Implement the **network initialization**, **forward pass**, and **backward pass**.

Throughout this assignment, you will be implementing layers in csdc618_dscd604/layers.py. You can re-use your implementations for affine_forward, affine_backward, relu_forward, relu_backward, and softmax_loss from before.

For now, don't worry about implementing dropout or batch/layer normalization yet, as you will add those features later.

```
[2]: # Setup cell.
import time
import numpy as np
import matplotlib.pyplot as plt
from csdc618_dscd604.classifiers.fc_net import *
from csdc618_dscd604.data_utils import get_CIFAR10_data
from csdc618_dscd604.gradient_check import eval_numerical_gradient, \
    eval_numerical_gradient_array
from csdc618_dscd604.solver import Solver

%matplotlib inline
plt.rcParams["figure.figsize"] = (10.0, 8.0) # Set default size of plots.
plt.rcParams["image.interpolation"] = "nearest"
plt.rcParams["image.cmap"] = "gray"

%load_ext autoreload
%autoreload 2

def rel_error(x, y):
    """Returns relative error."""
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))
```

```
[3]: # Load the (preprocessed) CIFAR-10 data.
data = get_CIFAR10_data()
for k, v in list(data.items()):
    print(f"{k}: {v.shape}")
```

```
X_train: (49000, 3, 32, 32)
y_train: (49000,)
X_val: (1000, 3, 32, 32)
y_val: (1000,)
```

```
X_test: (1000, 3, 32, 32)
y_test: (1000,)
```

1.1 Initial Loss and Gradient Check

As a sanity check, run the following to check the initial loss and to gradient check the network both with and without regularization.

This is a good way to see if the initial losses seem reasonable.

For gradient checking, you should expect to see errors around $1e-7$ or less.

```
[4]: np.random.seed(231)
N, D, H1, H2, C = 2, 15, 20, 30, 10
X = np.random.randn(N, D)
y = np.random.randint(C, size=(N,))

for reg in [0, 3.14]:
    print("Running check with reg = ", reg)
    model = FullyConnectedNet(
        [H1, H2],
        input_dim=D,
        num_classes=C,
        reg=reg,
        weight_scale=5e-2,
        dtype=np.float64
    )

    loss, grads = model.loss(X, y)
    print("Initial loss: ", loss)

    # Most of the errors should be on the order of e-7 or smaller.
    # NOTE: It is fine however to see an error for W2 on the order of e-5
    # for the check when reg = 0.0
    for name in sorted(grads):
        f = lambda _: model.loss(X, y)[0]
        grad_num = eval_numerical_gradient(f, model.params[name],
        ↪ verbose=False, h=1e-5)
        print(f"{name} relative error: {rel_error(grad_num, grads[name])}")
```

```
Running check with reg = 0
Initial loss: 2.3004790897684924
W1 relative error: 1.4839894098713283e-07
W2 relative error: 2.21204793107852e-05
W3 relative error: 3.527252851540647e-07
b1 relative error: 5.376386228531692e-09
b2 relative error: 2.085654200257447e-09
b3 relative error: 5.7957243458479405e-11
Running check with reg = 3.14
```

```
Initial loss: 7.052114776533016
W1 relative error: 3.904541941902138e-09
W2 relative error: 6.86942277940646e-08
W3 relative error: 2.131129848945024e-08
b1 relative error: 1.4752427965311745e-08
b2 relative error: 1.7223751746766738e-09
b3 relative error: 1.5702714832602802e-10
```

As another sanity check, make sure your network can *overfit* on a small dataset of 50 images.

First, we will try a three-layer network with 100 units in each hidden layer.

In the following cell, tweak the **learning rate** and **weight initialization scale** to overfit and achieve 100% training accuracy within 20 epochs.

```
[5]: # TODO: Use a three-layer Net to overfit 50 training examples by
      # tweaking just the learning rate and initialization scale.
```

```
num_train = 50
small_data = {
    "X_train": data["X_train"][:num_train],
    "y_train": data["y_train"][:num_train],
    "X_val": data["X_val"],
    "y_val": data["y_val"],
}

weight_scale = 1e-1    # tuned to overfit
learning_rate = 1e-3    # tuned to overfit

model = FullyConnectedNet(
    [100, 100],
    weight_scale=weight_scale,
    dtype=np.float64
)
solver = Solver(
    model,
    small_data,
    print_every=10,
    num_epochs=20,
    batch_size=25,
    update_rule="sgd",
    optim_config={"learning_rate": learning_rate},
)
solver.train()

plt.plot(solver.loss_history)
plt.title("Training loss history")
plt.xlabel("Iteration")
```

```
plt.ylabel("Training loss")
plt.grid(linestyle='--', linewidth=0.5)
plt.show()
```

(Iteration 1 / 40) loss: 357.428290

(Epoch 0 / 20) train acc: 0.220000; val_acc: 0.111000

(Epoch 1 / 20) train acc: 0.380000; val_acc: 0.141000

(Epoch 2 / 20) train acc: 0.520000; val_acc: 0.138000

(Epoch 3 / 20) train acc: 0.740000; val_acc: 0.130000

(Epoch 4 / 20) train acc: 0.820000; val_acc: 0.153000

(Epoch 5 / 20) train acc: 0.860000; val_acc: 0.175000

(Iteration 11 / 40) loss: 6.726589

(Epoch 6 / 20) train acc: 0.940000; val_acc: 0.163000

(Epoch 7 / 20) train acc: 0.960000; val_acc: 0.166000

(Epoch 8 / 20) train acc: 0.960000; val_acc: 0.164000

(Epoch 9 / 20) train acc: 0.980000; val_acc: 0.162000

(Epoch 10 / 20) train acc: 0.980000; val_acc: 0.162000

(Iteration 21 / 40) loss: 0.800243

(Epoch 11 / 20) train acc: 1.000000; val_acc: 0.158000

(Epoch 12 / 20) train acc: 1.000000; val_acc: 0.158000

(Epoch 13 / 20) train acc: 1.000000; val_acc: 0.158000

(Epoch 14 / 20) train acc: 1.000000; val_acc: 0.158000

(Epoch 15 / 20) train acc: 1.000000; val_acc: 0.158000

(Iteration 31 / 40) loss: 0.000000

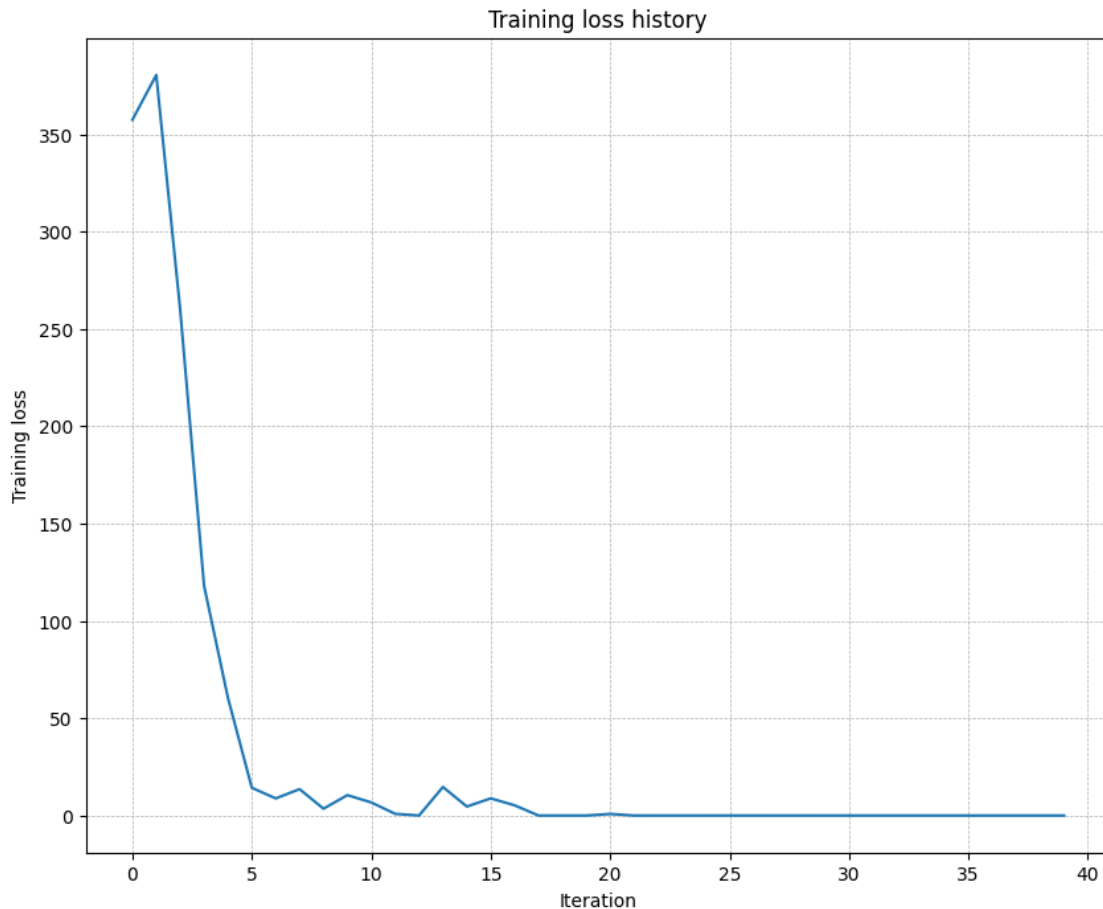
(Epoch 16 / 20) train acc: 1.000000; val_acc: 0.158000

(Epoch 17 / 20) train acc: 1.000000; val_acc: 0.158000

(Epoch 18 / 20) train acc: 1.000000; val_acc: 0.158000

(Epoch 19 / 20) train acc: 1.000000; val_acc: 0.158000

(Epoch 20 / 20) train acc: 1.000000; val_acc: 0.158000



Now, try to use a five-layer network with 100 units on each layer to overfit on 50 training examples.

Again, you will have to adjust the learning rate and weight initialization scale, but you should be able to achieve 100% training accuracy within 20 epochs.

[6]: *# TODO: Use a five-layer Net to overfit 50 training examples by
tweaking just the learning rate and initialization scale.*

```
num_train = 50
small_data = {
    'X_train': data['X_train'][:num_train],
    'y_train': data['y_train'][:num_train],
    'X_val': data['X_val'],
    'y_val': data['y_val'],
}

learning_rate = 1e-3 # tuned to overfit
weight_scale = 1e-1 # tuned to overfit
```

```

model = FullyConnectedNet(
    [100, 100, 100, 100],
    weight_scale=weight_scale,
    dtype=np.float64
)
solver = Solver(
    model,
    small_data,
    print_every=10,
    num_epochs=20,
    batch_size=25,
    update_rule='sgd',
    optim_config={'learning_rate': learning_rate},
)
solver.train()

plt.plot(solver.loss_history)
plt.title('Training loss history')
plt.xlabel('Iteration')
plt.ylabel('Training loss')
plt.grid(linestyle='--', linewidth=0.5)
plt.show()

```

```

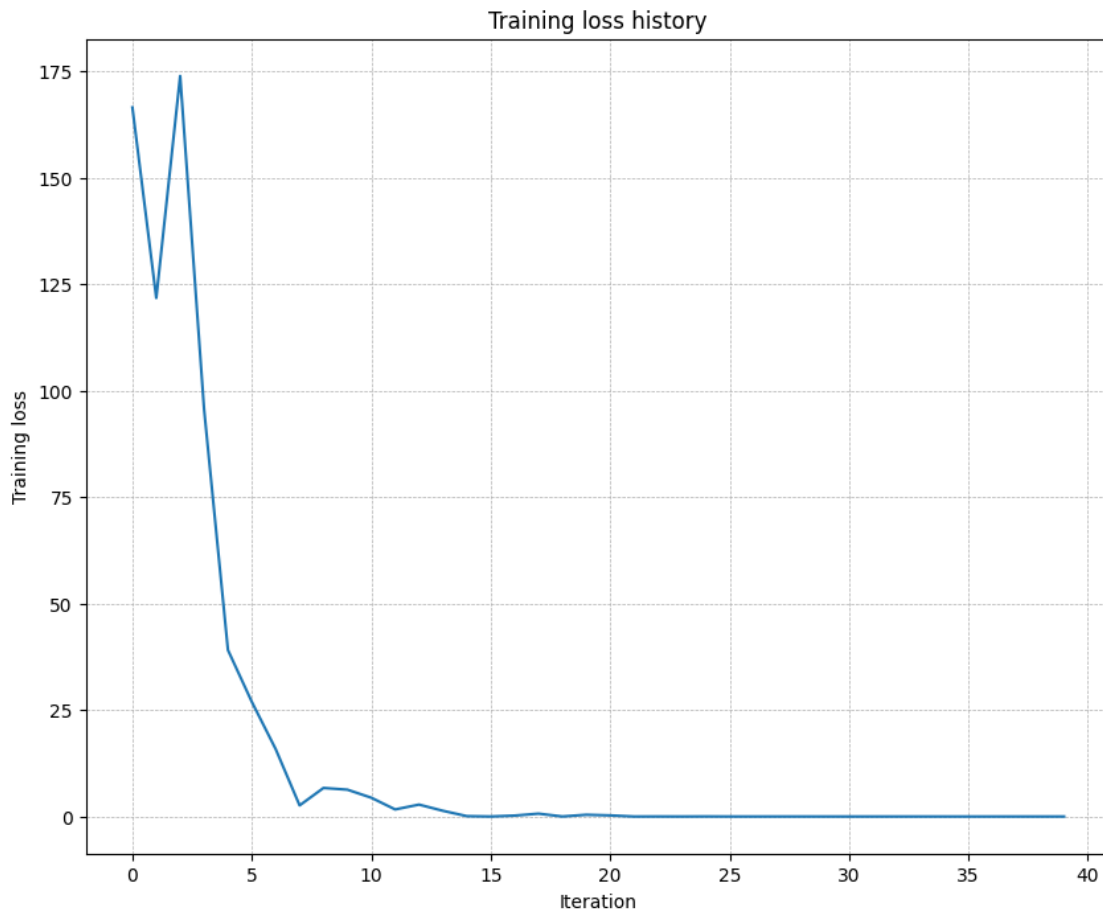
(Iteration 1 / 40) loss: 166.501707
(Epoch 0 / 20) train acc: 0.220000; val_acc: 0.116000
(Epoch 1 / 20) train acc: 0.240000; val_acc: 0.083000
(Epoch 2 / 20) train acc: 0.160000; val_acc: 0.104000
(Epoch 3 / 20) train acc: 0.520000; val_acc: 0.106000
(Epoch 4 / 20) train acc: 0.700000; val_acc: 0.131000

(Epoch 5 / 20) train acc: 0.700000; val_acc: 0.116000
(Iteration 11 / 40) loss: 4.414592
(Epoch 6 / 20) train acc: 0.840000; val_acc: 0.114000
(Epoch 7 / 20) train acc: 0.880000; val_acc: 0.108000
(Epoch 8 / 20) train acc: 0.900000; val_acc: 0.109000
(Epoch 9 / 20) train acc: 0.960000; val_acc: 0.114000
(Epoch 10 / 20) train acc: 0.980000; val_acc: 0.127000
(Iteration 21 / 40) loss: 0.261098
(Epoch 11 / 20) train acc: 1.000000; val_acc: 0.126000

(Epoch 12 / 20) train acc: 1.000000; val_acc: 0.124000
(Epoch 13 / 20) train acc: 1.000000; val_acc: 0.124000
(Epoch 14 / 20) train acc: 1.000000; val_acc: 0.124000
(Epoch 15 / 20) train acc: 1.000000; val_acc: 0.125000
(Iteration 31 / 40) loss: 0.000594
(Epoch 16 / 20) train acc: 1.000000; val_acc: 0.125000
(Epoch 17 / 20) train acc: 1.000000; val_acc: 0.125000

```


(Epoch 18 / 20) train acc: 1.000000; val_acc: 0.125000
(Epoch 19 / 20) train acc: 1.000000; val_acc: 0.125000
(Epoch 20 / 20) train acc: 1.000000; val_acc: 0.125000



1.1.1 Inline question 1:

Did you notice anything about the comparative difficulty of training the three-layer network vs. training the five-layer network?

In particular, based on your experience, which network seemed more sensitive to the initialization scale? Why do you think that is the case?

Your Answer: The five-layer network trained harder than the three-layer network. The five-layer network showed more sensitivity to the weight initialization scale. A deep network passes signals through more layers, so a poor scale shrinks or grows the activations and gradients at each step and the effect compounds with depth. The shallow network has fewer layers, so the same scale drifts less. The deep network needs a scale near the right range to keep forward activations and backward gradients stable.

2 Update rules

So far we have used vanilla stochastic gradient descent (SGD) as our update rule. More sophisticated update rules can make it easier to train deep networks.

We will implement a few of the most commonly used update rules and compare them to vanilla SGD.

2.1 SGD+Momentum

Stochastic gradient descent with momentum is a widely used update rule that tends to make deep networks converge faster than vanilla stochastic gradient descent.

Open the file `csdc618_dscd604/optim.py` and read the documentation at the top of the file to make sure you understand the API.

Implement the SGD+momentum update rule in the function `sgd_momentum` and run the following to check your implementation. You should see errors less than $e-8$.

```
[7]: from csdc618_dscd604.optim import sgd_momentum

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
v = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)

config = {"learning_rate": 1e-3, "velocity": v}
next_w, _ = sgd_momentum(w, dw, config=config)

expected_next_w = np.asarray([
    [ 0.1406,      0.20738947,  0.27417895,  0.34096842,  0.40775789],
    [ 0.47454737,  0.54133684,  0.60812632,  0.67491579,  0.74170526],
    [ 0.80849474,  0.87528421,  0.94207368,  1.00886316,  1.07565263],
    [ 1.14244211,  1.20923158,  1.27602105,  1.34281053,  1.4096      ]])
expected_velocity = np.asarray([
    [ 0.5406,      0.55475789,  0.56891579,  0.58307368,  0.59723158],
    [ 0.61138947,  0.62554737,  0.63970526,  0.65386316,  0.66802105],
    [ 0.68217895,  0.69633684,  0.71049474,  0.72465263,  0.73881053],
    [ 0.75296842,  0.76712632,  0.78128421,  0.79544211,  0.8096      ]])

# Should see relative errors around e-8 or less
print("next_w error: ", rel_error(next_w, expected_next_w))
print("velocity error: ", rel_error(expected_velocity, config["velocity"]))
```

```
next_w error:  8.882347033505819e-09
velocity error: 4.269287743278663e-09
```

Once you have done so, run the following to train a six-layer network with both SGD and SGD+momentum. You should see the SGD+momentum update rule converge faster.

```

[8]: num_train = 4000
small_data = {
    'X_train': data['X_train'][:num_train],
    'y_train': data['y_train'][:num_train],
    'X_val': data['X_val'],
    'y_val': data['y_val'],
}

solvers = {}

for update_rule in ['sgd', 'sgd_momentum']:
    print('Running with ', update_rule)
    model = FullyConnectedNet(
        [100, 100, 100, 100, 100],
        weight_scale=5e-2
    )

    solver = Solver(
        model,
        small_data,
        num_epochs=5,
        batch_size=100,
        update_rule=update_rule,
        optim_config={'learning_rate': 5e-3},
        verbose=True,
    )
    solvers[update_rule] = solver
    solver.train()

fig, axes = plt.subplots(3, 1, figsize=(15, 15))

axes[0].set_title('Training loss')
axes[0].set_xlabel('Iteration')
axes[1].set_title('Training accuracy')
axes[1].set_xlabel('Epoch')
axes[2].set_title('Validation accuracy')
axes[2].set_xlabel('Epoch')

for update_rule, solver in solvers.items():
    axes[0].plot(solver.loss_history, label=f"loss_{update_rule}")
    axes[1].plot(solver.train_acc_history, label=f"train_acc_{update_rule}")
    axes[2].plot(solver.val_acc_history, label=f"val_acc_{update_rule}")

for ax in axes:
    ax.legend(loc="best", ncol=4)
    ax.grid(linestyle='--', linewidth=0.5)

```

```
plt.show()
```

Running with `sgd`

```
(Iteration 1 / 200) loss: 2.559978
(Epoch 0 / 5) train acc: 0.104000; val_acc: 0.107000

(Iteration 11 / 200) loss: 2.356070

(Iteration 21 / 200) loss: 2.214091
(Iteration 31 / 200) loss: 2.205928

(Epoch 1 / 5) train acc: 0.225000; val_acc: 0.193000

(Iteration 41 / 200) loss: 2.132095
(Iteration 51 / 200) loss: 2.118950
(Iteration 61 / 200) loss: 2.116443

(Iteration 71 / 200) loss: 2.132549
(Epoch 2 / 5) train acc: 0.298000; val_acc: 0.260000
(Iteration 81 / 200) loss: 1.977227

(Iteration 91 / 200) loss: 2.007528
(Iteration 101 / 200) loss: 2.004762
(Iteration 111 / 200) loss: 1.885342

(Epoch 3 / 5) train acc: 0.343000; val_acc: 0.287000
(Iteration 121 / 200) loss: 1.891516
(Iteration 131 / 200) loss: 1.923677
(Iteration 141 / 200) loss: 1.957743

(Iteration 151 / 200) loss: 1.966736
(Epoch 4 / 5) train acc: 0.322000; val_acc: 0.305000
(Iteration 161 / 200) loss: 1.801483
(Iteration 171 / 200) loss: 1.973779

(Iteration 181 / 200) loss: 1.666572
(Iteration 191 / 200) loss: 1.909494
(Epoch 5 / 5) train acc: 0.372000; val_acc: 0.319000
Running with sgd_momentum
```

```
(Iteration 1 / 200) loss: 3.153778
(Epoch 0 / 5) train acc: 0.099000; val_acc: 0.088000
(Iteration 11 / 200) loss: 2.227203

(Iteration 21 / 200) loss: 2.125706
(Iteration 31 / 200) loss: 1.932679
(Epoch 1 / 5) train acc: 0.308000; val_acc: 0.258000

(Iteration 41 / 200) loss: 1.946330
(Iteration 51 / 200) loss: 1.780464
(Iteration 61 / 200) loss: 1.753502

(Iteration 71 / 200) loss: 1.844626
(Epoch 2 / 5) train acc: 0.377000; val_acc: 0.331000
```

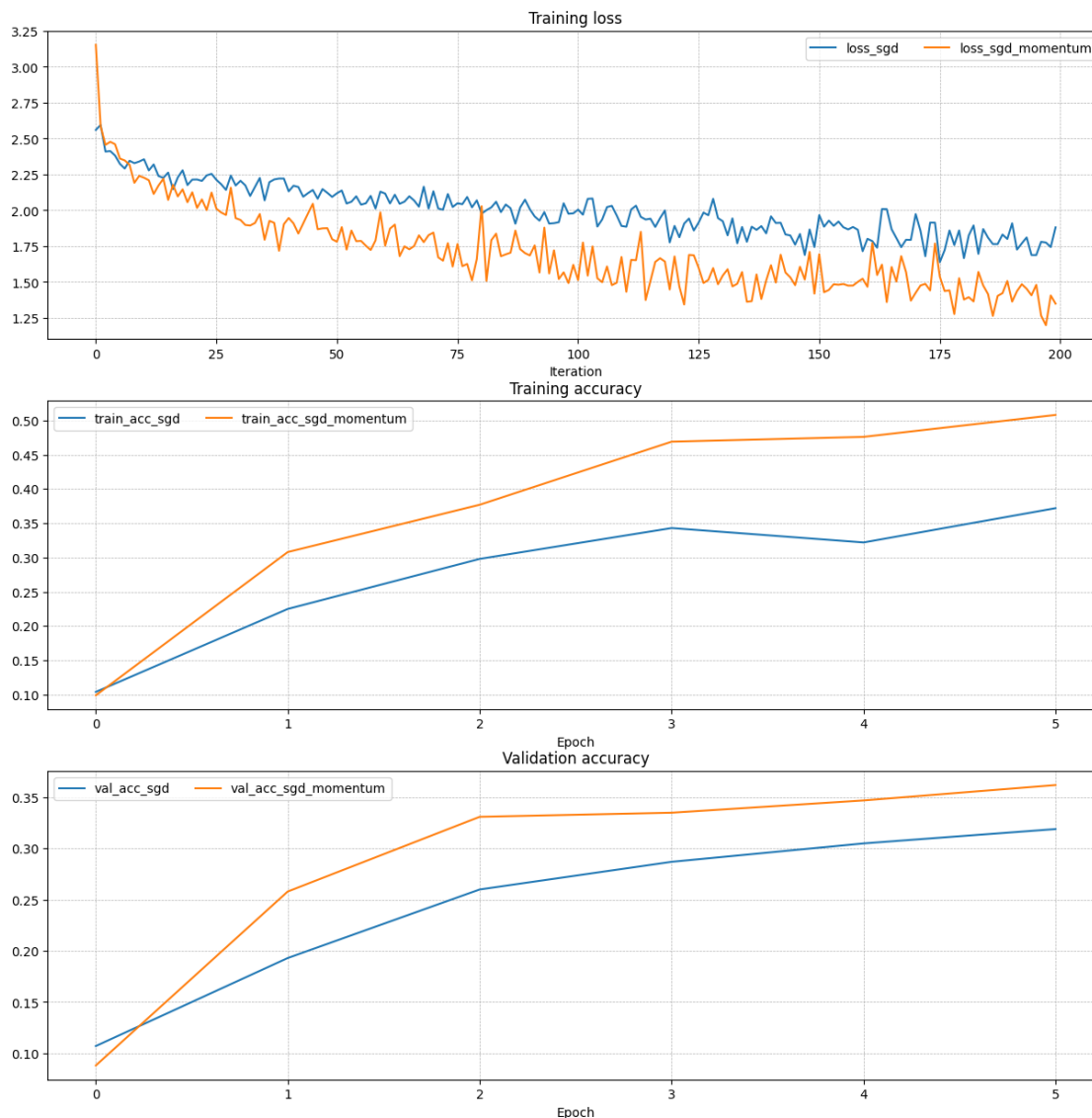
(Iteration 81 / 200) loss: 2.028389
(Iteration 91 / 200) loss: 1.685416
(Iteration 101 / 200) loss: 1.513204
(Iteration 111 / 200) loss: 1.431671

(Epoch 3 / 5) train acc: 0.469000; val_acc: 0.335000
(Iteration 121 / 200) loss: 1.678510
(Iteration 131 / 200) loss: 1.545366
(Iteration 141 / 200) loss: 1.617381

(Iteration 151 / 200) loss: 1.693830
(Epoch 4 / 5) train acc: 0.476000; val_acc: 0.347000
(Iteration 161 / 200) loss: 1.466674

(Iteration 171 / 200) loss: 1.425210
(Iteration 181 / 200) loss: 1.377764

(Iteration 191 / 200) loss: 1.363371
(Epoch 5 / 5) train acc: 0.508000; val_acc: 0.362000



2.2 RMSProp and Adam

RMSProp [1] and Adam [2] are update rules that set per-parameter learning rates by using a running average of the second moments of gradients.

In the file `csdc618_dscd604/optim.py`, implement the RMSProp update rule in the `rmsprop` function and implement the Adam update rule in the `adam` function, and check your implementations using the tests below.

NOTE: Please implement the *complete* Adam update rule (with the bias correction mechanism), not the first simplified version mentioned in the course notes.

[1] Tijmen Tieleman and Geoffrey Hinton. “Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude.” COURSERA: Neural Networks for Machine Learning 4 (2012).

[2] Diederik Kingma and Jimmy Ba, “Adam: A Method for Stochastic Optimization”, ICLR 2015.

```
[9]: # Test RMSProp implementation
from csdc618_dscd604.optim import rmsprop

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
cache = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)

config = {'learning_rate': 1e-2, 'cache': cache}
next_w, _ = rmsprop(w, dw, config=config)

expected_next_w = np.asarray([
    [-0.39223849, -0.34037513, -0.28849239, -0.23659121, -0.18467247],
    [-0.132737, -0.08078555, -0.02881884, 0.02316247, 0.07515774],
    [0.12716641, 0.17918792, 0.23122175, 0.28326742, 0.33532447],
    [0.38739248, 0.43947102, 0.49155973, 0.54365823, 0.59576619]])
expected_cache = np.asarray([
    [0.5976, 0.6126277, 0.6277108, 0.64284931, 0.65804321],
    [0.67329252, 0.68859723, 0.70395734, 0.71937285, 0.73484377],
    [0.75037008, 0.7659518, 0.78158892, 0.79728144, 0.81302936],
    [0.82883269, 0.84469141, 0.86060554, 0.87657507, 0.8926]])

# You should see relative errors around e-7 or less
print('next_w error: ', rel_error(expected_next_w, next_w))
print('cache error: ', rel_error(expected_cache, config['cache']))
```

```
next_w error: 9.524687511038133e-08
cache error: 2.6477955807156126e-09
```

```
[10]: # Test Adam implementation
from csdc618_dscd604.optim import adam

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
m = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)
v = np.linspace(0.7, 0.5, num=N*D).reshape(N, D)

config = {'learning_rate': 1e-2, 'm': m, 'v': v, 't': 5}
next_w, _ = adam(w, dw, config=config)

expected_next_w = np.asarray([
    [-0.40094747, -0.34836187, -0.29577703, -0.24319299, -0.19060977],
    [-0.1380274, -0.08544591, -0.03286534, 0.01971428, 0.0722929],
    [0.1248705, 0.17744702, 0.23002243, 0.28259667, 0.33516969],
    [0.38774145, 0.44031188, 0.49288093, 0.54544852, 0.59801459]])
```

```

expected_v = np.asarray([
    [ 0.69966,      0.68908382,   0.67851319,   0.66794809,   0.65738853,],
    [ 0.64683452,   0.63628604,   0.6257431,    0.61520571,   0.60467385,],
    [ 0.59414753,   0.58362676,   0.57311152,   0.56260183,   0.55209767,],
    [ 0.54159906,   0.53110598,   0.52061845,   0.51013645,   0.49966,   ]])
expected_m = np.asarray([
    [ 0.48,          0.49947368,   0.51894737,   0.53842105,   0.55789474],
    [ 0.57736842,   0.59684211,   0.61631579,   0.63578947,   0.65526316],
    [ 0.67473684,   0.69421053,   0.71368421,   0.73315789,   0.75263158],
    [ 0.77210526,   0.79157895,   0.81105263,   0.83052632,   0.85         ]])

# You should see relative errors around e-7 or less
print('next_w error: ', rel_error(expected_next_w, next_w))
print('v error: ', rel_error(expected_v, config['v']))
print('m error: ', rel_error(expected_m, config['m']))

```

```

next_w error:  1.1395691798535431e-07
v error:  4.208314038113071e-09
m error:  4.214963193114416e-09

```

Once you have debugged your RMSProp and Adam implementations, run the following to train a pair of deep networks using these new update rules:

```

[11]: learning_rates = {'rmsprop': 1e-4, 'adam': 1e-3}
for update_rule in ['adam', 'rmsprop']:
    print('Running with ', update_rule)
    model = FullyConnectedNet(
        [100, 100, 100, 100, 100],
        weight_scale=5e-2
    )
    solver = Solver(
        model,
        small_data,
        num_epochs=5,
        batch_size=100,
        update_rule=update_rule,
        optim_config={'learning_rate': learning_rates[update_rule]},
        verbose=True
    )
    solvers[update_rule] = solver
    solver.train()
    print()

fig, axes = plt.subplots(3, 1, figsize=(15, 15))

axes[0].set_title('Training loss')
axes[0].set_xlabel('Iteration')
axes[1].set_title('Training accuracy')

```



```

axes[1].set_xlabel('Epoch')
axes[2].set_title('Validation accuracy')
axes[2].set_xlabel('Epoch')

for update_rule, solver in solvers.items():
    axes[0].plot(solver.loss_history, label=f"{update_rule}")
    axes[1].plot(solver.train_acc_history, label=f"{update_rule}")
    axes[2].plot(solver.val_acc_history, label=f"{update_rule}")

for ax in axes:
    ax.legend(loc='best', ncol=4)
    ax.grid(linestyle='--', linewidth=0.5)

plt.show()

```

```

Running with adam
(Iteration 1 / 200) loss: 3.476928
(Epoch 0 / 5) train acc: 0.126000; val_acc: 0.110000

(Iteration 11 / 200) loss: 2.027711

(Iteration 21 / 200) loss: 2.183357
(Iteration 31 / 200) loss: 1.744257

(Epoch 1 / 5) train acc: 0.363000; val_acc: 0.330000
(Iteration 41 / 200) loss: 1.707951

(Iteration 51 / 200) loss: 1.703834
(Iteration 61 / 200) loss: 2.094758

(Iteration 71 / 200) loss: 1.505558
(Epoch 2 / 5) train acc: 0.419000; val_acc: 0.362000

(Iteration 81 / 200) loss: 1.594429
(Iteration 91 / 200) loss: 1.519017

(Iteration 101 / 200) loss: 1.368523
(Iteration 111 / 200) loss: 1.470400

(Epoch 3 / 5) train acc: 0.460000; val_acc: 0.378000
(Iteration 121 / 200) loss: 1.199064
(Iteration 131 / 200) loss: 1.464705

(Iteration 141 / 200) loss: 1.359863
(Iteration 151 / 200) loss: 1.415069

(Epoch 4 / 5) train acc: 0.521000; val_acc: 0.374000
(Iteration 161 / 200) loss: 1.382818
(Iteration 171 / 200) loss: 1.359900

(Iteration 181 / 200) loss: 1.095948
(Iteration 191 / 200) loss: 1.243088

```

(Epoch 5 / 5) train acc: 0.572000; val_acc: 0.382000

Running with rmsprop

(Iteration 1 / 200) loss: 2.589166

(Epoch 0 / 5) train acc: 0.119000; val_acc: 0.146000

(Iteration 11 / 200) loss: 2.032921

(Iteration 21 / 200) loss: 1.897278

(Iteration 31 / 200) loss: 1.770793

(Epoch 1 / 5) train acc: 0.381000; val_acc: 0.320000

(Iteration 41 / 200) loss: 1.895731

(Iteration 51 / 200) loss: 1.681091

(Iteration 61 / 200) loss: 1.486923

(Iteration 71 / 200) loss: 1.628511

(Epoch 2 / 5) train acc: 0.423000; val_acc: 0.341000

(Iteration 81 / 200) loss: 1.506182

(Iteration 91 / 200) loss: 1.600674

(Iteration 101 / 200) loss: 1.478501

(Iteration 111 / 200) loss: 1.577709

(Epoch 3 / 5) train acc: 0.487000; val_acc: 0.355000

(Iteration 121 / 200) loss: 1.495931

(Iteration 131 / 200) loss: 1.525799

(Iteration 141 / 200) loss: 1.552580

(Iteration 151 / 200) loss: 1.654283

(Epoch 4 / 5) train acc: 0.524000; val_acc: 0.362000

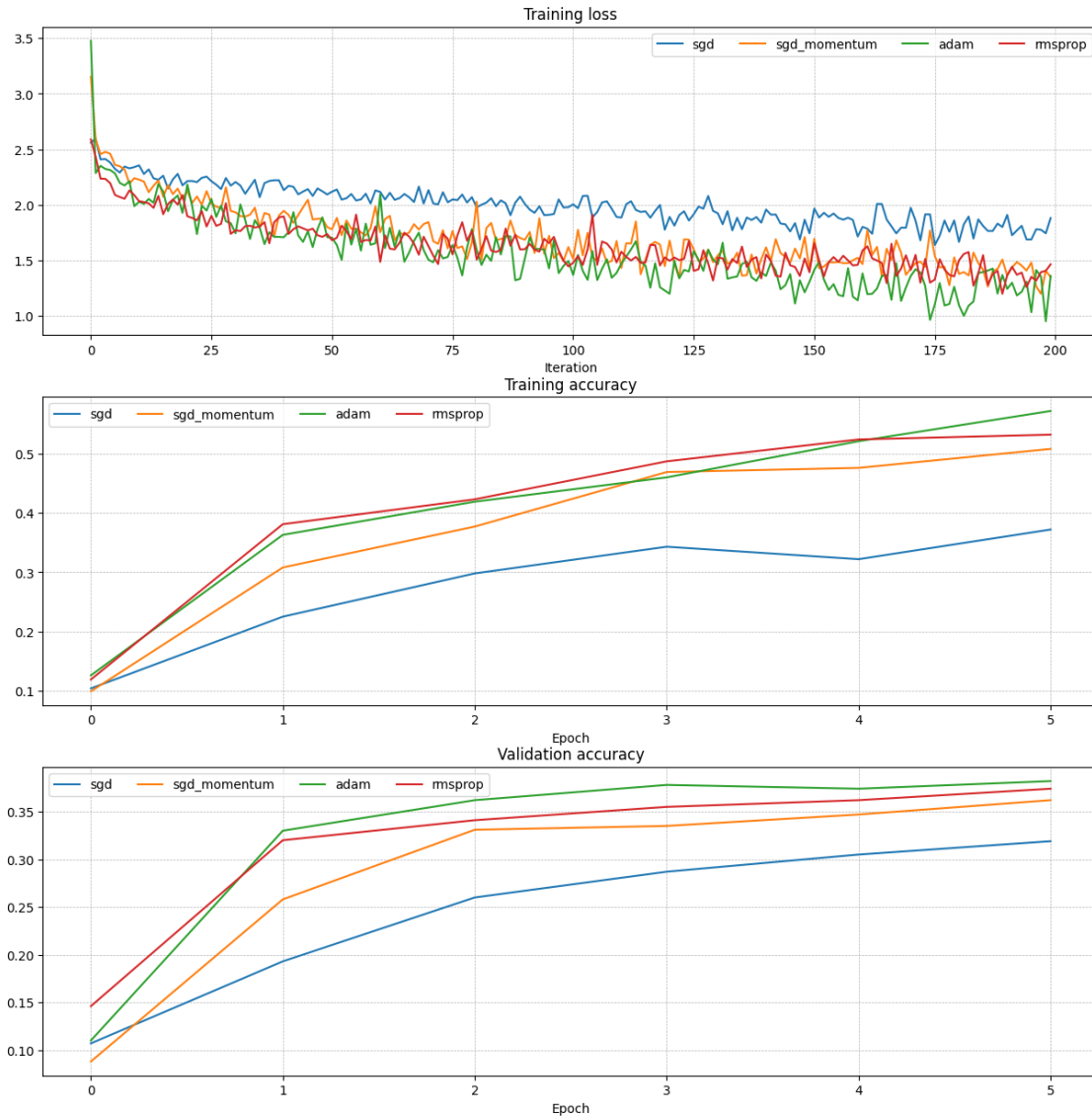
(Iteration 161 / 200) loss: 1.589372

(Iteration 171 / 200) loss: 1.413528

(Iteration 181 / 200) loss: 1.500273

(Iteration 191 / 200) loss: 1.365942

(Epoch 5 / 5) train acc: 0.532000; val_acc: 0.374000



2.3 Inline Question 2:

AdaGrad, like Adam, is a per-parameter optimization method that uses the following update rule:

```
cache += dw**2
w += - learning_rate * dw / (np.sqrt(cache) + eps)
```

If you notice that during training a network with AdaGrad the updates became very small, and that the network is learning slowly. Using your knowledge of the AdaGrad update rule, why do you think the updates would become very small? Would Adam have the same issue?

Your Answer: AdaGrad adds the squared gradient into the cache at every step and never decays the cache. The cache grows without bound, so the denominator $\sqrt{\text{cache}} + \epsilon$ keeps rising and the effective step size falls toward zero. Learning stalls. Adam avoids the issue. Adam uses an

exponentially decaying average of squared gradients, so the denominator stays bounded and the step size stays useful through training.

3 Train a Good Model!

Train the best fully connected model that you can on CIFAR-10, storing your best model in the `best_model` variable.

We require you to get at least 50% accuracy on the validation set using a fully connected network.

If you are careful it should be possible to get accuracies above 55%, but we don't require it for this part and won't assign extra credit for doing so.

```
[12]: best_model = None

#####
# TODO: Train the best FullyConnectedNet that you can on CIFAR-10. You might #
# find batch/layer normalization and dropout useful. Store your best model in #
# the best_model variable.                                                    #
#####
# *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
best_model = FullyConnectedNet(
    [256, 128],
    weight_scale=2e-2,
    reg=1e-2,
    dtype=np.float32,
)
solver = Solver(
    best_model, data,
    update_rule='adam',
    optim_config={'learning_rate': 1e-3},
    lr_decay=0.95, num_epochs=20, batch_size=200,
    verbose=True, print_every=200,
)
solver.train()
# *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
#####
#                                     END OF YOUR CODE                        #
#####
```

```
(Iteration 1 / 4900) loss: 6.272682
(Epoch 0 / 20) train acc: 0.196000; val_acc: 0.200000

(Iteration 201 / 4900) loss: 2.311366
(Epoch 1 / 20) train acc: 0.441000; val_acc: 0.430000

(Iteration 401 / 4900) loss: 1.960521
```

(Epoch 2 / 20) train acc: 0.457000; val_acc: 0.457000
(Iteration 601 / 4900) loss: 1.652360
(Epoch 3 / 20) train acc: 0.471000; val_acc: 0.432000
(Iteration 801 / 4900) loss: 1.763128
(Epoch 4 / 20) train acc: 0.486000; val_acc: 0.423000
(Iteration 1001 / 4900) loss: 1.701782
(Iteration 1201 / 4900) loss: 1.666672
(Epoch 5 / 20) train acc: 0.481000; val_acc: 0.449000
(Iteration 1401 / 4900) loss: 1.673032
(Epoch 6 / 20) train acc: 0.517000; val_acc: 0.488000
(Iteration 1601 / 4900) loss: 1.633453
(Epoch 7 / 20) train acc: 0.497000; val_acc: 0.483000
(Iteration 1801 / 4900) loss: 1.500456
(Epoch 8 / 20) train acc: 0.509000; val_acc: 0.481000
(Iteration 2001 / 4900) loss: 1.625396
(Iteration 2201 / 4900) loss: 1.484643
(Epoch 9 / 20) train acc: 0.539000; val_acc: 0.494000
(Iteration 2401 / 4900) loss: 1.636025
(Epoch 10 / 20) train acc: 0.522000; val_acc: 0.497000
(Iteration 2601 / 4900) loss: 1.442037
(Epoch 11 / 20) train acc: 0.547000; val_acc: 0.502000
(Iteration 2801 / 4900) loss: 1.487884
(Epoch 12 / 20) train acc: 0.519000; val_acc: 0.497000
(Iteration 3001 / 4900) loss: 1.409148
(Epoch 13 / 20) train acc: 0.548000; val_acc: 0.493000
(Iteration 3201 / 4900) loss: 1.438570
(Iteration 3401 / 4900) loss: 1.546652
(Epoch 14 / 20) train acc: 0.547000; val_acc: 0.506000
(Iteration 3601 / 4900) loss: 1.446806
(Epoch 15 / 20) train acc: 0.537000; val_acc: 0.482000
(Iteration 3801 / 4900) loss: 1.429947
(Epoch 16 / 20) train acc: 0.572000; val_acc: 0.514000

```
(Iteration 4001 / 4900) loss: 1.495496
(Epoch 17 / 20) train acc: 0.574000; val_acc: 0.532000
(Iteration 4201 / 4900) loss: 1.326993
(Iteration 4401 / 4900) loss: 1.318047
(Epoch 18 / 20) train acc: 0.584000; val_acc: 0.518000
(Iteration 4601 / 4900) loss: 1.335499
(Epoch 19 / 20) train acc: 0.596000; val_acc: 0.528000
(Iteration 4801 / 4900) loss: 1.343013
(Epoch 20 / 20) train acc: 0.596000; val_acc: 0.532000
```

4 Test Your Model!

Run your best model on the validation and test sets. You should achieve at least 50% accuracy on the validation set and the test set.

```
[13]: y_test_pred = np.argmax(best_model.loss(data['X_test']), axis=1)
      y_val_pred = np.argmax(best_model.loss(data['X_val']), axis=1)
      print('Validation set accuracy: ', (y_val_pred == data['y_val']).mean())
      print('Test set accuracy: ', (y_test_pred == data['y_test']).mean())
```

```
Validation set accuracy:  0.532
Test set accuracy:  0.508
```